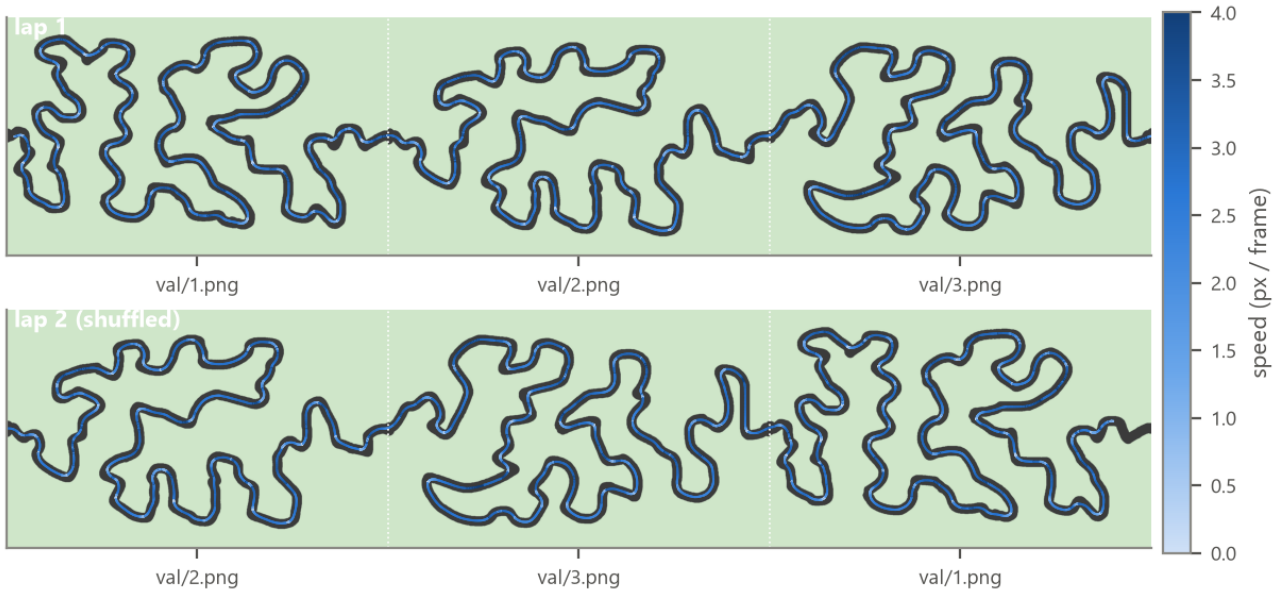
Car Evolution

Complete project guide

Neuro-evolution of car drivers: bit-string DNA → neural network → simulated car on a modular track, evolved by a self-adaptive genetic algorithm on an island graph.



A driver evolved by this project (your run *new_train_val_tracks_success*, file `bestValDNA.txt`) re-simulated on the validation track with the rules it was evolved under (before the 2026-10-10 changes). Colour = speed. It finishes all 6 screens in 136 simulated seconds.

What this guide covers	
How to run it, every file, and which files belong to the visual route vs. the optimised headless route	§2–§4
Tracks, the car's sensors, controls, physics, fitness and death rules — with real numbers	§5–§6
The genome, every chromosome → neural-network decoder (worked bit-level examples)	§7–§9
Initialisation, selection, crossover (traced example), mutation	§10–§13
Islands, migration, the generation loop, checkpoints and output files	§14–§15
What every graph shows, every parameter and what changing it does, caveats found while reading	§16–§19
Planned experiments: 17 neurons (% 17) and Gray decoding — what to change and what it does	App. C

Generated on 2026-10-10 by reading the code in `Car_evolution_extended` as it is on disk today. Every example number in this guide was computed by running copies of the project's own functions (decoders, crossover, network evaluation) or taken from your run folders — not hand-derived. **Code changes made later on 2026-10-10 are included:** the sensor step is one `SENSOR_STEP` parameter in main.py (1 px) used by all three simulators, the speed input is normalised to 0–1, backward connections are reversed instead of discarded, and the phenotypic diversity compares the edges as evaluated (§4, §6.2, §8.1, §9, §16.2).

Contents

1. The project in one page . . . 4

2. Running the project . . . 6

2.1 Command line . . . 6

2.2 Dependencies . . . 6

2.3 What a run produces . . . 6

3. File map . . . 7

4. The two execution routes (optimisation rework) . . . 8

4.1 Stage-by-stage comparison . . . 8

4.2 How to watch the cars drive . . . 9

4.3 The GA-glue optimisations (always on) . . . 9

5. Tracks . . . 10

5.1 Segments and how they are loaded . . . 10

5.2 Assembly into two laps . . . 10

5.3 Start position and joining rule . . . 11

5.4 Drawing new segments . . . 11

6. The car: sensors, control, physics, fitness . . . 11

6.1 State and coordinates . . . 11

6.2 Sensors (network inputs) . . . 11

6.3 Outputs → controls (normalize_pair) . . . 12

6.4 Movement and collision . . . 13

6.5 Fitness and death rules . . . 13

6.6 What the evolved cars actually do . . . 14

7. The genome . . . 15

7.1 Container classes . . . 15

7.2 Rate bits and self-adaptation (decode_rates) . . . 15

7.3 Size of a genome . . . 16

8. Decoders: from chromosome to neural network . . . 16

8.1 Conventions shared by all decoders . . . 16

8.2 connection_based_markers — the active decoder . . . 16

8.3 connection_based — fixed 24-bit genes . . . 18

8.4 matrix_connections — adjacency matrix + weight list . . . 18

8.5 triangular_matrix_connections . . . 19

8.6 fixed_topology — weights only . . . 19

8.7 grammar_matrix — an L-system that writes the matrix . . . 19

8.8 cellular_division — instruction list . . . 20

8.9 cellular_encoding — Gruau's developmental grammar . . . 20

8.10 Diploid decoders (Double_DNA_one_chromosome) . . . 21

9. Evaluating the network . . . 21

9.1 connection_based_sort_feed_forward (Computation.py) . . . 21

9.2 The dense equivalent used for training . . . 22

9.3 connection_based_max_range_propagation (recurrent, not active) . . . 22

10. Initialisation operators . . . 22

11. Selection . . . 23

11.1 tournament_selection (active) . . . 23

11.2 roulette_selection . . . 23

12. Crossover . . . 23

12.1 Common frame of every single-DNA crossover . . . 23

12.2 connection_based_markers — the active crossover . . . 23

12.3 Simple cut crossovers (legacy) . . . 24

12.4 Block-aligned crossovers for the other encodings (legacy) . . . 25

12.5 Diploid crossovers (legacy) . . . 25

13. Mutation . . . 26

13.1 The active chain . . . 26

13.2 What a given mutation rate means . . . 26

13.3 Structure-aware mutations (legacy) . . . 26

14. Population structure and the generation loop . . . 27

14.1 The loop, step by step (runEvolutionIslands) . . . 27

14.2 Island topologies . . . 27

14.3 Migration (_plan_migration) . . . 28

14.4 Building the next generation of an island (evolve_island) . . . 28

14.5 Single-population mode . . . 28

14.6 Fitness caching: the `new` flag . . . 28

14.7 Randomness and reproducibility . . . 28

15. Saving, checkpoints and resuming . . . 29

15.1 Checkpoint files . . . 29

15.2 Resuming a run . . . 29

15.3 bestDNA.txt and bestValDNA.txt . . . 29

16. Graphs and console output explained . . . 30

16.1 bestFitnessGraph.png . . . 30

16.2 diversityGraph.png . . . 31

16.3 islandTopology.png . . . 32

16.4 bestTrainNetwork.png and bestValNetwork.png . . . 32

16.5 Console output per generation . . . 33

17. Parameter reference . . . 33

17.1 main.py . . . 33

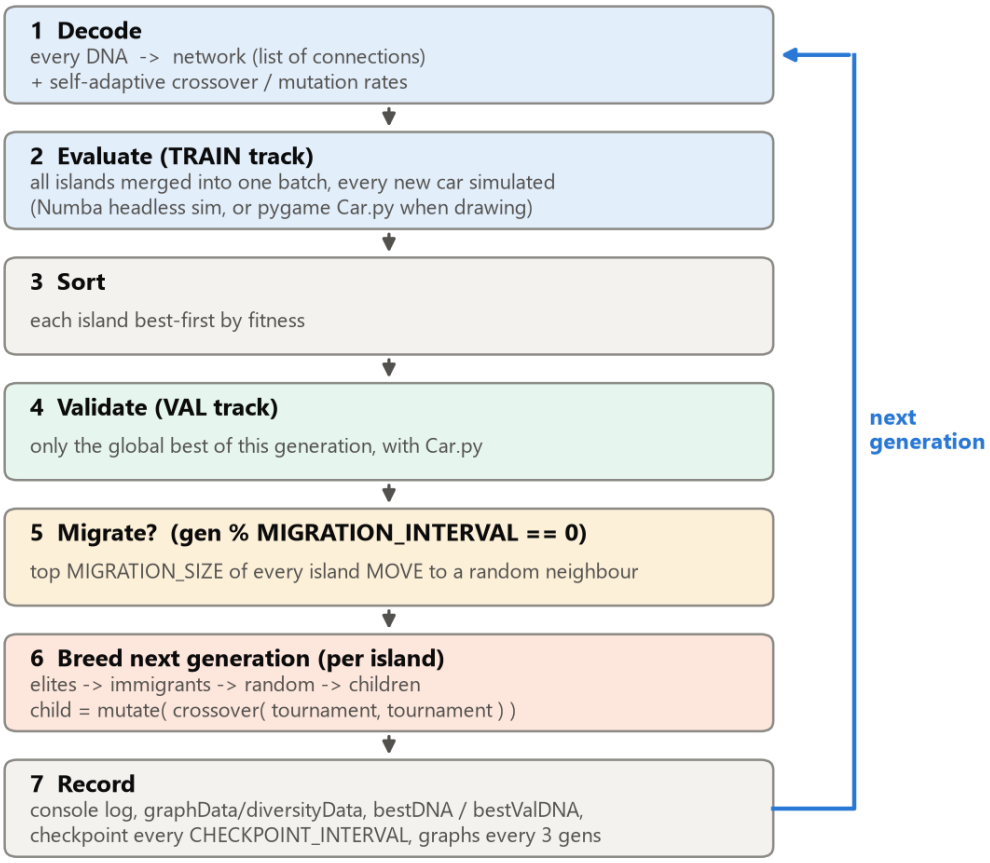
17.2 Constants inside EvolutionEngine.__init__ . . . 34

17.3 Car physics constants (keep in sync in three places) . . . 34
18. Legacy and auxiliary files . . . 35
18.1 test.py . . . 35
18.2 ideas.py . . . 35
18.3 scetch.txt, the CSV and old assets . . . 35
19. Caveats found while reading the code . . . 36
Appendix A — Recipes . . . 37
Appendix B — Polish UI terms . . . 37
Appendix C — Planned experiments: 17 neurons and Gray decoding . . . 38
17 neurons with % 17 . . . 38
Gray decoding . . . 39

1. The project in one page

The program evolves drivers for a small 2-D car seen from above. Each driver is a **neural network**, and each network is grown from a **DNA bit string**. Nobody trains the networks with gradients: a **genetic algorithm (GA)** keeps the DNA of drivers that get far along the track, recombines and mutates it, and repeats for thousands of generations.

- **Individual** = DNA (a numpy array of 0/1, ~5–10 thousand bits) + the network decoded from it + its fitness.
- **Network** = a list of weighted connections between up to 16 neurons: 6 inputs (5 distance sensors + speed), 6 hidden, 4 outputs (turn left, turn right, accelerate, brake).
- **Fitness** = how far right the car got on a long track made of hand-drawn screens (10 points per screen), plus a big time bonus if it reaches the end.
- **Population** = 1600 individuals split into 16 **islands** of 100 on a 4×4 grid. Islands evolve separately; every 50 generations the best individual of each island moves to a neighbour (**migration**).
- **Train / validation split**: selection only ever sees the train track. The best driver of every generation is also driven on a separate validation track to measure generalisation (the overfitting check).
- **Self-adaptation**: the first 32 bits of every DNA encode that individual's own crossover rate and mutation rate, so the GA tunes its own operators.



One generation of the active configuration (island mode). Step numbers are used throughout the guide.

How the pieces are wired

`main.py` does not contain algorithms; it picks **operator callables** from the operator classes and hands them to `EvolutionEngine`. The engine only ever calls them, so any part can be swapped in `main.py`:

```
DNA_INITIALIZATION = Initializer().init_single_DNA_one_chromosome(INPUTS, OUTPUTS, MARKER) \
                         .random_bits
DNA_DECODER        = Decoder().decodes_single_DNA_one_chromosome(INPUTS, OUTPUTS, MARKER) \
                         .connection_based_markers
COMPUTATION        = Computation(INPUTS, OUTPUTS, MARKER).connection_based_sort_feed_forward
SELECTION          = Selection().tournament_selection
CROSSOVER          = Crossover().crossover_single_DNA_one_chromosome(_MARKER=MARKER) \
                         .connection_based_markers
MUTATION           = MutationCombiner([...random_bit_flip, ...random_insert,
                                       ...random_delete]).mutate
```

Callable	Signature as used by the engine	Called in
DNA_INITIALIZATION	`() -> DNA object` (e.g. Single_DNA_one_chromosome)	createIslandPopulations, random injection
DNA_DECODER	`(dnaType) -> nn` (list of {source, target, weight})	step 1, every individual, every generation
COMPUTATION	`(nn, input_vector[6]) -> outputs[4]`	Car.py only (visual route + validation)
SELECTION	`(island: list[Individual]) -> Individual`	step 6, twice per child
CROSSOVER	`(parent1, parent2) -> child`	step 6, once per child
MUTATION	`(child) -> child`	step 6, once per child

2. Running the project

2.1 Command line

```
cd Car_evolution_extended          # paths 'assets/...' and 'evolutionOutput/...' are relative
.venv\Scripts\python.exe main.py 42   # 42 = random seed (REQUIRED: main.py reads sys.argv[1])
```

- The seed initialises both `random` and `numpy.random`; the Numba crossover PRNG is seeded once from numpy. Same seed + same code + same screen size → same run (see §14.7).
- A pygame window (90 % of your screen) opens even in headless mode. It stays green; its title bar shows the validation progress. Everything else is printed to the console and written to `evolutionOutput/<date_time>/`.
- The first generation is slower: Numba compiles the simulation and DNA kernels (cached in `__pycache__` afterwards).
- Stop with Ctrl+C or by closing the window. A run can be resumed from its last checkpoint (§15.2).

2.2 Dependencies

`requirements.txt` lists numpy, pygame, matplotlib and opencv-python. **numba is not listed** but the optimised route uses it; without numba the engine silently falls back to the much slower pure-NumPy simulation. Your `.venv` currently has Python 3.13.11, numba 0.65.1, numpy 2.4.5, pygame 2.6.1, matplotlib 3.10.9 and OpenCV 4.13.

2.3 What a run produces

File in evolutionOutput/<run>/	Written	Content
parameters.txt	once at start	window size, population, start points, island settings, sensor step, neighbour lists
graphData.txt	every generation	one row per generation: gen, best, mean, val, mean crossover rate, mean mutation rate, best of island 0..15
diversityData.txt	every generation	gen + diversity of every island
bestDNA.txt	every generation	DNA of the current generation's global best (overwritten)
bestValDNA.txt	on new val record	DNA of the individual that set the best validation score so far
bestFitnessGraph.png, diversityGraph.png, islandTopology.png	every 3 generations	graphs (§16)
bestTrainNetwork.png, bestValNetwork.png	every 3 generations	drawings of the best networks (§16.4)
checkpoints/gen_000100.txt …	every 100 generations	every individual's DNA, per island (§15)

3. File map

Route legend: **shared** = used whatever you choose; **visual** = per-car pygame route (also used for validation); **headless** = optimised route; **GA-fast** = the array/Numba operators from the optimisation rework; **legacy** = not used by the current configuration.

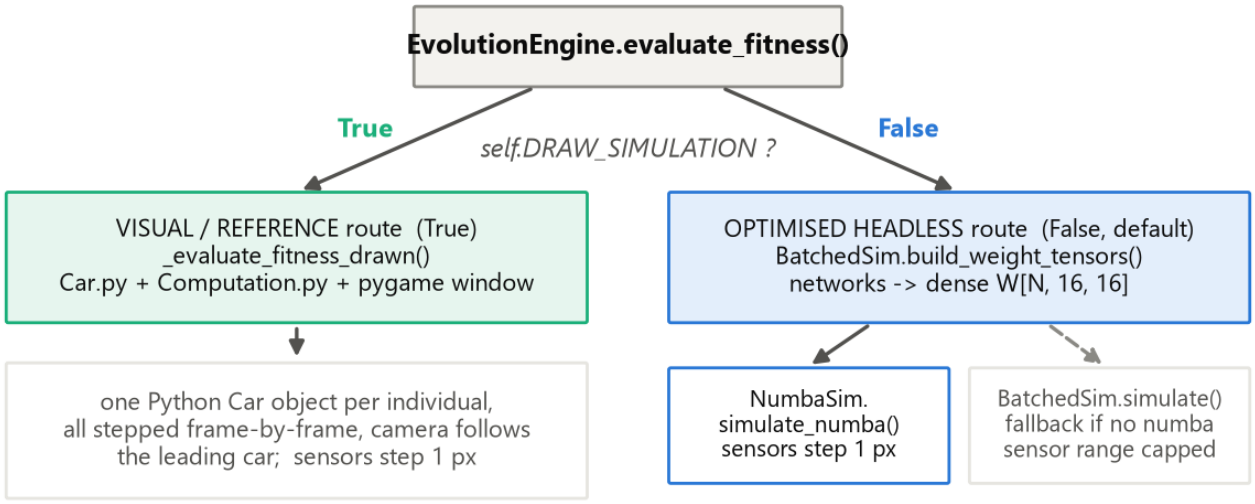
File	What is inside	Route
main.py	Experiment configuration (all tunable parameters + chosen operators). Builds EvolutionEngine and calls run().	shared (entry point)
EvolutionEngine.py	The orchestrator: window, track loading/drawing/assembly, populations and islands, the evaluation dispatch, validation, migration, breeding, logging, graphs, diversity, checkpoints.	shared — contains the branch between the visual and headless routes
Individual.py	One individual: dnaType, nn, fitness, testFitness, new flag, crossover_rate, mutation_rate; clone().	shared (clone = GA-fast)
DNA.py	DNA containers: Single_DNA_one_chromosome (active), Double_DNA_one_chromosome (diploid, with combine()), multi-chromosome stubs.	shared
DNA_Initialization.py	Initializer: random genomes for every encoding, single and double.	shared; random_bits = GA-fast
DNA_Decoder.py	decode_rates() + Decoder: 8 single-chromosome decoders and 15 diploid ones.	shared; connection_based_markers → FastDNA
Computation.py	Network evaluation for the per-car route: sorted feed-forward (active) and recurrent propagation.	visual + validation
Selection.py	tournament_selection (active), roulette_selection.	shared
Crossover.py	All crossovers (simple cuts, weighted, gene-aware, diploid).	shared; connection_based_markers → FastDNA
Mutation.py	MutationCombiner, bit flip / insert / delete, structure-aware and diploid mutations.	shared; flip/insert/delete → FastDNA
FastDNA.py	Numba / NumPy kernels: decode_markers, crossover_markers, mutate_bit_flip/insert/delete.	GA-fast
Car.py	Car physics, sensors, controls, collision, fitness; pygame sprites and drawing.	visual + validation
NumbaSim.py	simulate_numba: whole population, one car per CPU thread, compiled.	headless (primary)
BatchedSim.py	build_weight_tensors (always used headless) and simulate (pure-NumPy fallback).	headless
Colors.py	RGB constants. GREEN (0, 255, 0) is the wall colour.	shared
utils.py	bits_to_int, int_to_bits, Gray-code helpers (unused), generateRandomDna, connectionToDNA.	shared
test.py	Post-hoc evaluation of 4 seeds on test tracks with metrics — written for the old circular track.	legacy, out of date (§18)
ideas.py	Prototypes: Gray-coded marker decoder, diploid organism, homologous crossover, alternative network runners. Not imported.	legacy / notes
scetch.txt	Research plan and notes (Polish).	notes
dna_marker_simulation_extended.csv	Measured gene density in random DNA for marker lengths 2–8 (§8.2).	analysis data
assets/train/, assets/val/	Track segments used by the run (10 train, 3 validation images).	shared
assets/car_sprite.png, car_dead_sprite.png	Car sprites (alive / crashed).	visual + validation
assets/bestDNA.txt	Loaded only when LOAD_MODEL = True.	optional

File	What is inside	Route
assets/parameters.txt, assets/test_track*.png	Start positions and tracks for test.py (circular-track era).	legacy
assets/old/, assets/old_segs/	Archive of older tracks and segments; never loaded.	legacy

4. The two execution routes (optimisation rework)

The optimisation rework changed two independent things. Keeping them apart makes the code much easier to read:

1. **Fitness evaluation** became headless and compiled. This is a real **switch** with two routes, chosen by `self.DRAW_SIMULATION` in `EvolutionEngine.__init__` (line 50, currently `False`).
2. **The GA glue** (decoding, copying, crossover, mutation, diversity) became array-based. This is **not a switch**: it is always on, but only for the specific operators that were rewritten (the GA-fast set below).



Both routes return one fitness per individual. Validation of the global best ALWAYS uses Car.py (drawn only when DRAW_SIMULATION=True). Decoding, selection, crossover and mutation are shared by both routes.

The branch inside `evaluate_fitness()`. Everything before (decoding) and after (sorting, breeding) is identical for both routes.

4.1 Stage-by-stage comparison

Stage	Visual / reference route (DRAW_SIMULATION = True)	Optimised headless route (False — default)
Decode DNA	DNA_Decoder → FastDNA.decode_markers (same)	same
Network form	list of dicts, evaluated by `Computation.connection_based_sort_feed_forward` every frame for every car	`BatchedSim.build_weight_tensors` packs all networks into one dense array W[N, 16, 16] (backward edges reversed, self-loops dropped, duplicates summed)
Simulation	`_evaluate_fitness_drawn`: one `Car` object per individual, all stepped together frame by frame in Python	`NumbaSim.simulate_numba`: compiled, `prange` = one car per CPU thread; fallback `BatchedSim.simulate` (NumPy) if numba is missing
Rendering	pygame: camera follows the right-most living car; sprites; optional sensor rays; graphs also shown in OpenCV windows	none; graphs are only saved as PNG files

Stage	Visual / reference route (DRAW_SIMULATION = True)	Optimised headless route (False — default)
Sensor ray step	`SENSOR_STEP` from main.py (default 1 px), passed to every Car	the same `SENSOR_STEP`, passed to simulate_numba / simulate — one parameter for all routes
Ray length	unlimited (march until wall)	unlimited in Numba; capped at MAX_SENSOR_DIST = 1 screen width in the NumPy fallback (1 536 samples per ray at 1-px steps: memory-heavy for large populations)
Frame limit	none (death timers only)	400 000 frames (Numba), 300 000 (NumPy fallback)
Validation of the best	Car.py, drawn	Car.py, not drawn (still per-frame Python, but only one car)
Speed	slow: watch-only, use small populations	fast: Numba measured at roughly 1 500× the per-car route when it was introduced (with 4-px sensor steps; 1-px steps make the ray marching up to ~4× more work)

Note
Since 2026-10-10 all three simulators take the sensor step from one parameter (`SENSOR_STEP` in main.py, passed through EvolutionEngine), use the same normalised speed input and the same edge handling. Checked with SENSOR_STEP = 1 and 3 on 33 networks on the validation track: Car.py, NumbaSim and the NumPy fallback give the same fitness to within 1.4e-11 (those cars all crashed early, so very long runs can still drift apart through floating-point summation order).

Caveat
The `COMPUTATION` callable from `main.py` is **only** used by Car.py. The headless training simulation always runs its own built-in feed-forward. If you switch `COMPUTATION` to the recurrent `connection_based_max_range_propagation`, training still uses feed-forward while validation uses the recurrent version.

4.2 How to watch the cars drive

1. Set `self.DRAW_SIMULATION = True` in `EvolutionEngine.__init__`.
2. Optional: set `self.DRAW_SENSORS = True` in `Car.__init__` to draw the 5 rays (white lines, magenta hit points).
3. Use a small population (for example `POPULATION_SIZE = 160` with 16 islands of 10, or `NUM_SUBPOPULATIONS = 1`): every car is a Python object that is updated and drawn every frame.

The window title then shows: generation, number of living cars (*Żywych*), simulated time (*Czas* = frames / 120) and the best fitness so far in this generation. Crashed cars stay on screen with the dark sprite.

4.3 The GA-glue optimisations (always on)

Change	Where	Why it is faster
DNA is a numpy uint8 array end-to-end	Initializer.random_bits, FastDNA	no list ↔ array conversion per operator; O(L) vector ops
Individual.clone() instead of deepcopy	Individual.py, used by crossover/migration/elites	copies only the DNA array (.copy()) and scalars; the network is not copied (it is re-decoded every generation anyway)
Tournament returns a reference	Selection.tournament_selection	parents are never modified in place (every operator returns a new array)
Shallow island copies	runEvolutionIslands (top of the loop)	list(island) instead of deepcopy
Compiled marker decoder	FastDNA.decode_markers (Numba)	bit scan in native code
Compiled gene-aware crossover	FastDNA.crossover_markers (Numba, own PRNG seeded once from numpy)	native loop over both parents
Vectorised mutations	FastDNA.mutate_bit_flip / insert / delete (NumPy)	one random vector per genome instead of a Python loop per bit

Change	Where	Why it is faster
Cheap diversity	_diversity_genotypic (per-locus sums), Jaccard subsampled to 600	no N×L matrix, no $O(N^2)$ over 1600
Graphs every 3 generations	GRAPH_INTERVAL = 3	matplotlib drawing dominated small populations
Fitness caching via `new`	evaluate_fitness (both routes)	individuals carried over unchanged are not re-simulated

Fast-path operators (the only ones that went through the rework): `Initializer.random_bits`, `Decoder.connection_based_markers`, `Crossover.connection_based_markers`, `Mutation.random_bit_flip`, `Mutation.random_insert`, `Mutation.random_delete`. Every other initializer, decoder, crossover and mutation is the original pure-Python code. They still exist and most still work, but some are incompatible with numpy DNA (§19).

Not optimised: the single-population loop `runEvolution` (still uses `deepcopy` every generation), validation (one per-frame Python car), file writing and graph drawing.

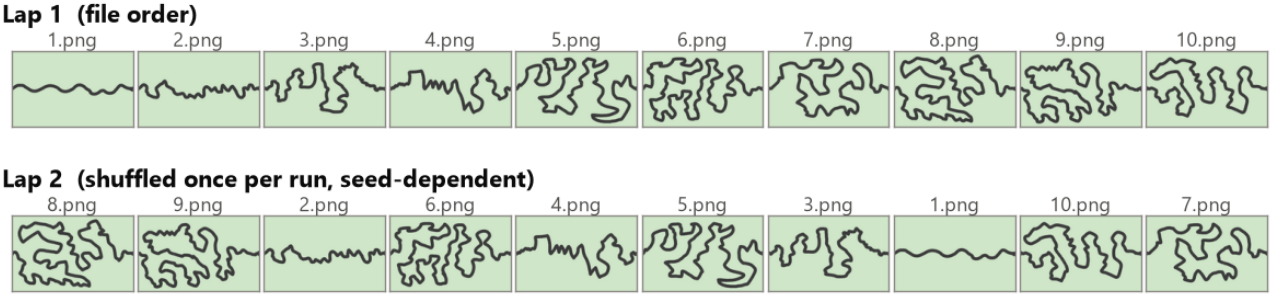
5. Tracks

5.1 Segments and how they are loaded

- A track is a list of **segments** = images in `assets/train/` or `assets/val/` (`.png .jpg .jpeg .bmp`). Each segment is scaled to the window size (WINDOW_WIDTH × WINDOW_HEIGHT = 90 % of your display; 1536 × 960 on this machine).
- **Colour meaning:** a pixel that is exactly pure green `(0, 255, 0)` is a wall; **everything else is road**. The drawing tool paints black. A slightly different green (e.g. anti-aliased edges from another paint program) is road.
- **Order:** file names are sorted with a dotted-decimal rule (`_segment_sort_key`): 1 < 1.5 < 1.7 < 2 < 10 < 10.3. To insert a segment between 5 and 6, name it `5.5.png` — nothing else needs renaming.
- **Disable without deleting:** a file whose name starts with `x` or `X` is skipped (rename `7.png` → `x7.png`).
- Current assets: **train** = 10 segments (1.png is a gentle wave; later ones are tight mazes), **validation** = 3 segments.

5.2 Assembly into two laps

`assembleTrack` builds lap 1 from the segments in file order and lap 2 from the **same segments shuffled once** (`random.shuffle`, so the order is fixed for the whole run and determined by the seed). Total width = 2 × n × WINDOW_WIDTH: the train track is 20 screens (30 720 px), the validation track 6 screens (9 216 px). The collision map is one boolean numpy array `[x, y]` (True = wall) for the whole track; for drawing, segments stay separate surfaces so no giant pygame surface is ever created.



The 10 train segments in lap 1 order, and an example lap 2 shuffle (the actual shuffle depends on the seed). Walls drawn light green here; in the PNG files they are pure green.

5.3 Start position and joining rule

Both tracks start at x = 0.02 × WINDOW_WIDTH (30.7 px), y = WINDOW_HEIGHT / 2 (480 px), facing right (angle 0). Because segments are concatenated and shuffled, **every segment must have road at its left-centre and right-centre edge**; the drawing tool shows two red guide dots at exactly those points.

5.4 Drawing new segments

Set `DRAW_TRAIN_TRACK = True` and/or `DRAW_VAL_TRACK = True` in `main.py`. At start-up you draw one screen at a time:

Input	Action
left mouse drag	paint road (black disc of the current brush radius)
right mouse drag	erase (paint wall green)
mouse wheel	change brush radius; minimum MIN_BRUSH_SIZE = round(1.1 × CAR_HEIGHT) ≈ 8 px so corridors are never narrower than the car can pass; maximum 1000
Enter	save as `assets/<train|val>/<n>.png`, start the next segment
Esc	finish drawing; the run starts with the segments drawn in this session

> **Caveat**
> Numbering restarts at 1, so drawing **overwrites** existing `1.png`, `2.png`, … in that folder, and only the segments drawn in this session are used for the run (not the rest of the folder). Copy the folder first if you want to keep the old track.

6. The car: sensors, control, physics, fitness

Every frame a car does exactly this (Car.update, mirrored line by line in NumbaSim):

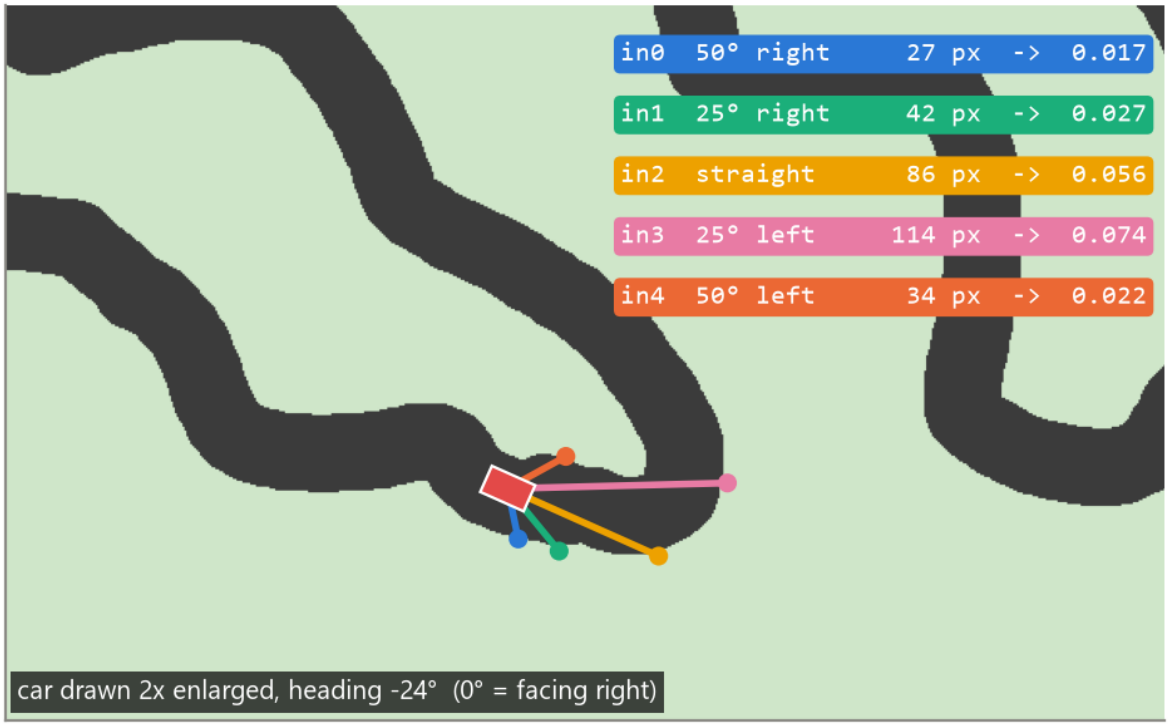
```
1  sensors      5 rays -> distances -> inputs in0..in4 (+ in5 = speed)
2  network      inputs -> outputs (turn left, turn right, accelerate, brake)
3  controls     outputs -> angle and speed changes
4  move         position += speed * direction
5  collision    5 body points against the wall map  -> dead?
6  fitness      progress, finish bonus, death timers
```

6.1 State and coordinates

- `position` (x, y) in pixels of the whole track; y grows downwards (screen coordinates).
- `angle` in degrees: 0 = facing right. The code moves along (cos θ, sin θ) with θ = radians(360 − angle), so a **larger angle turns the nose up = turn left**; `turn_left` adds, `turn_right` subtracts.
- `speed` in px per frame, clamped to [0, 4] (MINIMUM_SPEED = 0, MAX_SPEED = 4). Cars cannot reverse.
- Size: CAR_WIDTH = 0.008 × WINDOW_WIDTH ≈ 12.3 px, CAR_HEIGHT = 0.6 × CAR_WIDTH ≈ 7.4 px.
- Time: one call of `update` = one frame; all time constants are in frames; 120 frames = 1 simulated second.

6.2 Sensors (network inputs)

Five rays leave the car's centre at angle + (i − 2) × 25°, i.e. offsets −50°, −25°, 0°, +25°, +50°. A ray advances in steps of SENSOR_STEP pixels until its pixel is a wall or outside the track; the travelled length is the reading. Inputs are `length / SEGMENT_WIDTH` (1536 px), and the 6th input is **speed / MAX_SPEED** (0–1). The step is the single `SENSOR_STEP` parameter in main.py (default 1 px), used by Car.py, NumbaSim and BatchedSim alike.



A real frame of the evolved driver on the validation track (heading -24°). The car is entering a left bend: the right rays (in0, in1) are short, the 25° left ray (in3) sees far along the road. (Pixel lengths from the earlier 4-px-step simulation.)

Input neuron	0 (in0)	1 (in1)	2 (in2)	3 (in3)	4 (in4)	5 (in5)
Meaning	ray 50° right	ray 25° right	ray straight	ray 25° left	ray 50° left	speed / 4
Value in the figure	0.017	0.027	0.056	0.074	0.022	(speed)

Note the scales: in the maze segments a ray rarely sees more than a few hundred pixels (inputs < 0.2), while the speed input is speed / MAX_SPEED in 0–1. Until 2026-10-10 it was the raw speed (0–4), several times larger than every sensor input.

6.3 Outputs → controls (normalize_pair)

The network returns 4 numbers (each tanh, so in −1…+1): output 12 = turn left, 13 = turn right, 14 = accelerate, 15 = brake. They are used in two **pairs**, each converted to two non-negative forces that sum to 1 (or both 0):

Case	Rule	Example (a, b)	Forces
both > 0	proportional: a/(a+b), b/(a+b)	(0.15, 0.42)	(0.26, 0.74)
both < 0	negate and swap, then proportional — the less negative one wins	(−0.2, −0.6)	(0.75, 0.25)
mixed signs or a zero	shift so the smaller is 0 → winner takes all	(0.5, −0.3)	(1, 0)
		(0.0, 0.4)	(0, 1)
equal		(0.3, 0.3)	(0.50, 0.50)
both exactly 0 (unconnected outputs)		(0, 0)	(0, 0)

Then: `angle += 3·left − 3·right` (TURN_SPEED = 3°/frame, so the net turn is between −3° and +3° per frame); `speed = min(speed + 2·accelerate, 4)`; `speed = max(speed − 2·brake, 0)` (ACCELERATION = 2). Because the accelerate clamp happens before braking, a 50/50 pair at top speed slows the car (4 → 4 → 3).

Worked example with the hand-designed network drawn in §8.2 (left sensor → turn left, right sensor → turn right, straight sensor → accelerate, speed → brake, straight sensor → hidden 8 → brake), computed with the project's `Computation` class:

Situation	inputs in0..in4, speed	outputs 12 / 13 / 14 / 15	forces L/R, acc/brake	turn	new speed
road bends left	0.02, 0.04, 0.06, 0.12, 0.2; speed 2 → in5 = 0.5	0.537 / 0.060 / 0.235 / 0.943	0.90/0.10, 0.20/0.80	+2.40°	2 → 0.80
open straight	0.05, 0.09, 0.4, 0.09, 0.05; speed 3 → in5 = 0.75	0.149 / 0.149 / 0.922 / 0.932	0.50/0.50, 0.50/0.50	+0.00°	3 → 2.99

In the bend the left rays see more road, so turn-left wins (+2.4°/frame) and the brake output dominates (speed 2 → 0.80). On the straight, left and right cancel exactly and throttle and brake balance near the current speed (3 → 2.99). The speed → brake weight is +4 because the speed input is now 0–1.

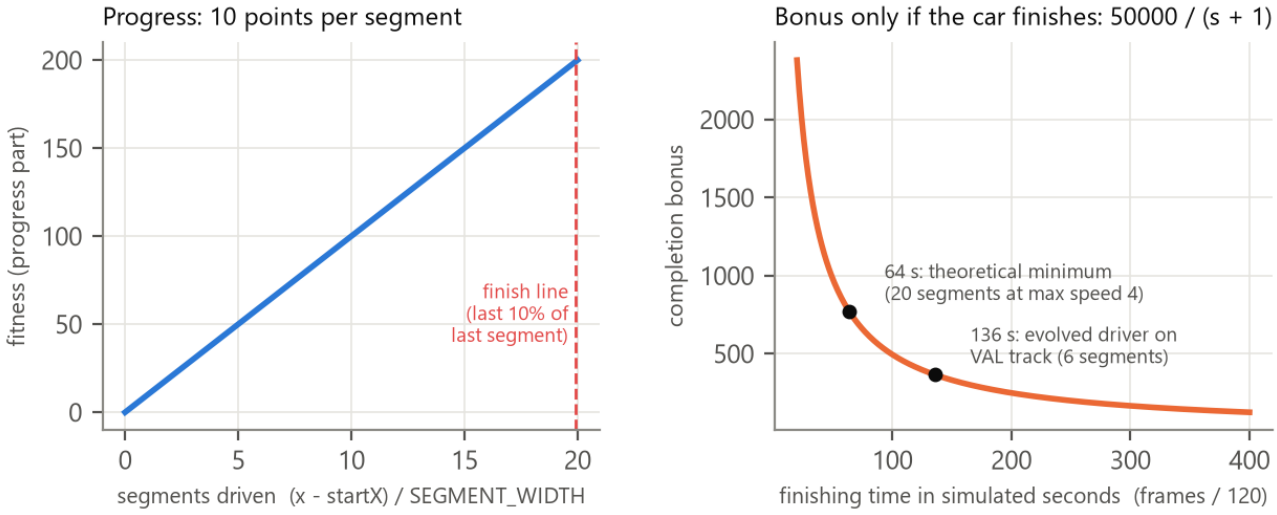
6.4 Movement and collision

After the controls, `x += cos θ · speed`, `y += sin θ · speed`. The car dies if its centre leaves the track or if any of 5 body points — the centre and the 4 corners of the rotated CAR_WIDTH × CAR_HEIGHT rectangle — lies on a wall pixel or outside the map. Collision is tested only at the new position each frame (no swept test), so with a top speed of 4 px/frame a wall thinner than ~4 px between two roads could in principle be jumped over.

6.5 Fitness and death rules

```
progress = (maxX - startX) / SEGMENT_WIDTH * 10        # 10 points per screen, maxX = furthest x ever reached
if maxX >= total_width - 0.1 * SEGMENT_WIDTH:           # reached the last 10 % of the last screen
    fitness = progress + 50000 / (frames / 120 + 1)     # finish bonus, larger when faster; car stops
else:
    fitness = progress
```

Death rule	Condition	Simulated time
crash	centre outside the track, or a body point on a wall / outside	immediate
no progress	maxX has not grown by ≥ 0.1 fitness units (≈ 15 px) within 12 000 frames	100 s
stalled	speed ≤ 0.001 for more than 1 200 frames	10 s
frame cap	400 000 frames (Numba) / 300 000 (NumPy fallback) / none (Car.py)	3 333 s
finished	reached the finish zone (fitness gets the bonus)	—



Left: the progress part grows linearly with distance. Right: the completion bonus as a function of finishing time — the bonus dominates fitness for every car that finishes.

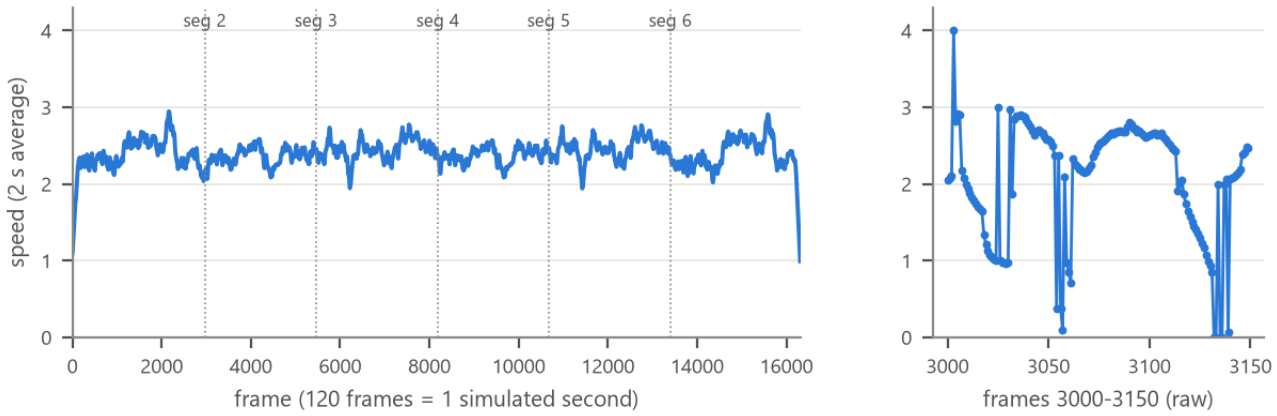
Key idea
Reading fitness values: on the **train** track (20 screens) progress tops out at 199.8, so any train fitness above ~200 means **the car finished** and the rest encodes its time (e.g. 340 ≈ finished in ~354 s). On the **validation** track (6 screens) the threshold is ~60: the evolved driver below scores 424 = 59.8 + 50000 / (136 + 1).

Because `maxX` only grows, driving backwards never lowers fitness; a car that circles in place dies after 100 s of no progress with the fitness it had.

6.6 What the evolved cars actually do

Re-simulating your best validation driver (run *new_train_val_tracks_success*, `bestValDNA.txt`: 7 260 bits, 73 genes, 35 forward connections) on the validation segments with the training simulator **as it was when the driver was evolved** (4-px sensor steps, raw speed input, backward edges discarded) gives fitness 423.97 in 16 311 frames — the log of that run recorded 426.64 with its own lap-2 shuffle.

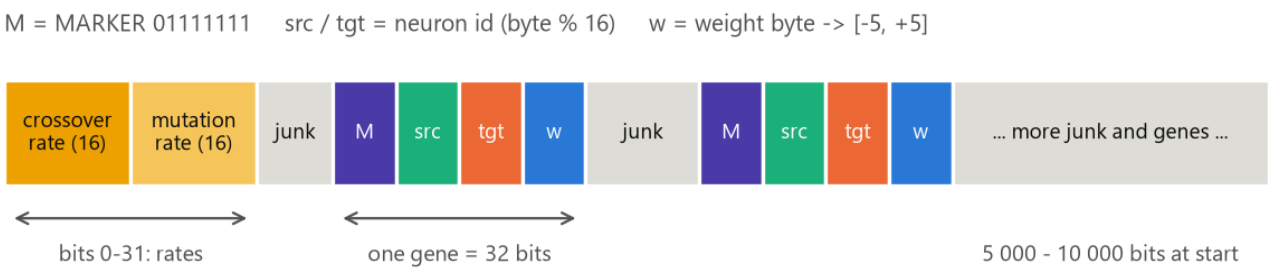
Caveat
Under the rules of 2026-10-10 (1-px sensors, speed input / 4, backward edges reversed) the same DNA decodes to a different effective network and crashes immediately (fitness 0.00 in both Car.py and NumbaSim). Drivers, bestValDNA files and checkpoints from earlier runs are not comparable with new runs and should not be resumed for a comparison.



Speed of the same drive. Left: 2-second moving average — it cruises at ~2.4 px/frame (60 % of the maximum). Right: 150 raw frames — throttle is almost bang-bang: the speed jumps between 0 and 4 within a few frames, braking hard before tight turns.

- It stays near the middle of the corridor by turning toward the side whose rays are longer.
- It never uses full speed for long: at 4 px/frame and 3°/frame turning, the tightest bends of these segments cannot be taken.
- It does not 'know' where it is on the track; it reacts only to the current 5 distances and speed (no memory: the feed-forward network has no state between frames).

7. The genome



Layout of a Single_DNA_one_chromosome with the active marker encoding.

7.1 Container classes

- `Single_DNA_one_chromosome.DNA` — one strand; a **numpy uint8 array** with the active initializer (the legacy initializers and checkpoint loading produce Python lists, which the fast operators also accept).
- `Double_DNA_one_chromosome.DNAa / .DNAb` — diploid: two strands plus `combine()` (§8.10).
- `Single_DNA_multi_chromosome`, `Double_DNA_multi_chromosome` — empty stubs, not used anywhere.

7.2 Rate bits and self-adaptation (decode_rates)

```
crossover_rate = int(DNA[0:16])  / 65536     # plain binary, linear -> [0, 1)
mutation_rate  = int(DNA[16:32]) / 65536
# genome shorter than 32 bits -> defaults (0.5, 0.005)
```

Example: bits `0100110011001100` = 19 660 → crossover rate 0.3000; bits `0000001010001111` = 655 → mutation rate 0.0100. The **crossover rate** is the probability that crossover happens at all (the mean of both parents' rates). The **mutation rate** is the per-bit probability used by each of the three mutations (flip, insert, delete). Both rates are inherited and mutated like any other bits, so selection tunes them.

Generation	Mean crossover rate	Mean mutation rate	Best train	Mean train	Val of the best
0	0.500	0.5000	0.0	0.0	0.0
25	0.384	0.2607	9.4	0.1	1.4
50	0.286	0.0979	222.7	5.1	122.4
100	0.181	0.0589	281.6	31.5	269.1
200	0.179	0.0525	296.7	102.9	308.5
400	0.282	0.0574	303.3	159.1	0.4
740	0.296	0.0531	342.9	169.7	426.6

From `graphData.txt` of your run *new_train_val_tracks_success* (row 0 is the placeholder). The mean mutation rate collapses from 0.5 to ~0.06 within 100 generations; before it does, almost nothing improves.

Key idea

The logged mean is dominated by outliers. Decoding all 1 600 genomes in that run's checkpoints gives a **median mutation rate of 0.0002 at generation 100 and 0.0001 at generations 400 and 700**, while the mean stays at ~0.057. Most individuals mutate ~1 bit per genome; a minority mutates heavily.

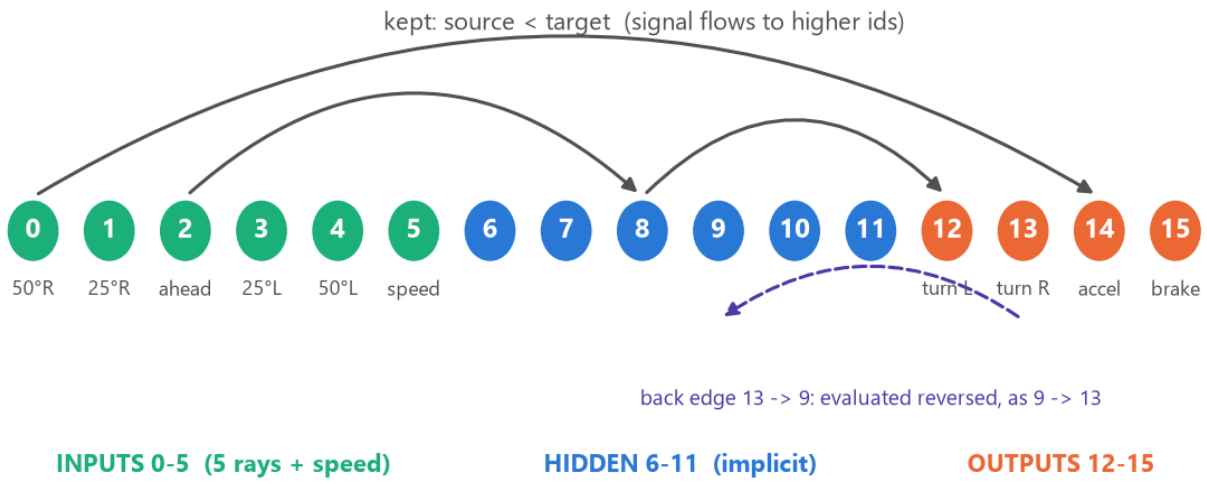
Why outliers keep appearing: (1) the mapping is linear binary, so the top bit alone is worth 0.5; (2) the rate field is not protected — an inserted or deleted bit anywhere in bits 0–31 shifts the whole field by one position (roughly doubling or halving the value, and the last crossover-rate bit becomes the first mutation-rate bit); (3) crossover mixes the 32 rate bits one by one, so a child's rate can be far from both parents'.

7.3 Size of a genome

`random_bits` creates 32 rate bits + a uniformly random 5 000–10 000 body bits. Measured over 200 fresh genomes: mean length 7 734 bits, mean 27.2 genes. Every gene except self-loops (1 in 16) is now used by the network — about 25.5 per genome; before backward edges were reversed only the ~12.5 forward ones were. Lengths then drift with insert/delete mutations: in generation 400 of your run they ranged from 172 to 10 028 bits (median 7 225) with a median of 48 genes.

8. Decoders: from chromosome to neural network

8.1 Conventions shared by all decoders



The neuron id space for INPUTS = [0..5], OUTPUTS = [12..15]. Hidden neurons are not declared: they are simply every other id below NR_OF_NEURONS = OUTPUTS[-1] + 1 = 16.

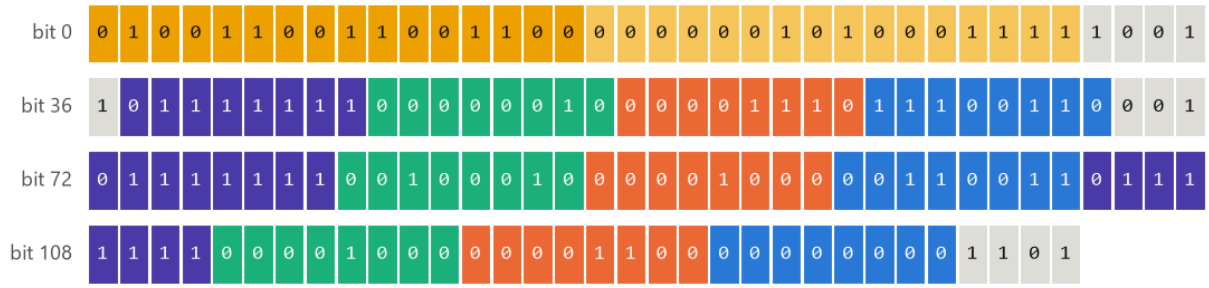
- **Network = list of connections** `{'source': s, 'target': t, 'weight': w}`. Duplicates, self-loops and backward edges may appear in the list; the evaluator drops self-loops, evaluates a backward edge t → s as s → t and sums duplicates (§9).
- **Ids from bytes:** source and target are an 8-bit value modulo 16. Because 256 is a multiple of 16 this is uniform, and it means only the low 4 bits of each byte matter — the high 4 bits are neutral and can mutate freely (for 17 neurons see Appendix C).
- **Weights from bytes:** `w = raw / 255 × 10 − 5`, i.e. 256 levels from −5 to +5 in steps of 0.039. raw 0 → −5, 127 → −0.02, 255 → +5.
- **Signal direction:** signal always flows from the lower to the higher id. A gene with source > target is not discarded but evaluated reversed with the same weight (13 → 9 acts as 9 → 13), so 15 of every 16 random genes are usable instead of 120 of 256. Inputs 0–5 can therefore feed anything; outputs can feed later outputs (12 → 14); a gene between two inputs (3 → 0, evaluated as 0 → 3) adds to the higher input (inputs are linear accumulators, no tanh).
- Every decoder skips bits 0–31 (the rate bits).

8.2 connection_based_markers — the active decoder

Genes are found by **searching for the marker** `01111111` instead of by position. Everything between genes is junk and ignored. Implemented in `FastDNA._decode_markers` (Numba); the Python decoder method only forwards to it.

```
i = 32
while i < len(DNA) - 31:
    if DNA[i:i+8] == MARKER:                    # 01111111
        src = int(DNA[i+8 :i+16]) % 16
        tgt = int(DNA[i+16:i+24]) % 16
        w   = int(DNA[i+24:i+32]) / 255 * 10 - 5
        emit(src, tgt, w)
        i += 32                                 # jump over the whole gene
    else:
        i += 1                                  # junk bit: slide by one
```

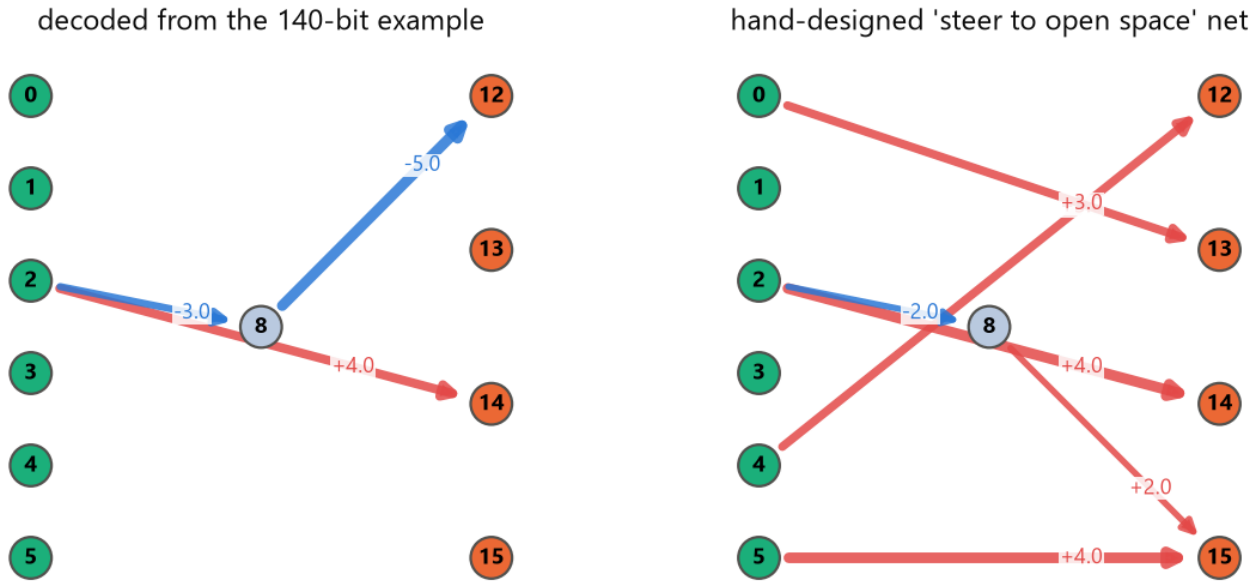
Worked example: a 140-bit genome



Colours: yellow = rate bits, grey = junk, violet = marker, green = source byte, orange = target byte, blue = weight byte. Each row is 36 bits.

Gene starts at bit	source byte	target byte	weight byte	Decoded connection
37	`00000010` = 2	`00001110` = 14	`11100110` = 230	2 → 14, w = +4.02
72	`00100010` = 34	`00001000` = 8	`00110011` = 51	2 → 8, w = -3.00 (34 % 16 = 2)
104	`00001000` = 8	`00001100` = 12	`00000000` = 0	8 → 12, w = -5.00

Rates: crossover 0.300, mutation 0.0100. Bits 32–36, 69–71 and 136–139 are junk.



Left: the decoded example network. Right: the hand-designed network used for the control example in §6.3.

This network is valid but a bad driver: with the road ahead open (in2 = 0.4) it gives hidden 8 = tanh(−1.2) = −0.83, turn-left = tanh(4.17) = 1.00 and accelerate = tanh(1.61) = 0.92, while turn-right and brake are unconnected (0). Both pairs are winner-takes-all, so the car turns left 3° every frame at full throttle — it spins and dies of 'no progress'.

Most random networks fail like this, which is why generation 0 scores almost nothing.

Why markers: robustness to insertions and deletions

Insert a single bit at position 33 (inside the first junk run) and decode again:

Decoder	Original genome	After inserting 1 bit at position 33
connection_based_markers	2→14 (+4.02), 2→8 (-3.00), 8→12 (-5.00)	2→14 (+4.02), 2→8 (-3.00), 8→12 (-5.00)
connection_based (fixed 24-bit frames)	11→8 (-4.37), 7→1 (-0.02), 2→8 (-3.00) …	13→12 (-4.69), 11→8 (+2.49), 1→4 (-4.02) …

The marker decoder returns exactly the same network. The fixed-frame decoder reads every 24 bits as a gene from bit 32 on, so it already decodes garbage across the junk, and one inserted bit changes every gene after it (a **frame shift**). Markers give the genome positional freedom: genes can move, duplicate, be separated by any amount of junk, and insert/delete mutations only damage the genes they actually hit.

How marker length controls gene density

Marker	Length	Genes in 10 M random bits	Probability per position	Share of DNA in genes
`01`	2	357,183	3.572 %	92.9 %
`011`	3	312,359	3.124 %	84.3 %
`0111`	4	249,809	2.498 %	69.9 %
`01111`	5	178,355	1.784 %	51.7 %
`011111`	6	113,062	1.131 %	33.9 %
`0111111`	7	65,376	0.654 %	20.3 %
`01111111`	8	35,561	0.356 %	11.4 %

From `dna_marker_simulation_extended.csv`. With the 8-bit marker a random genome contains a gene every ~280 bits (≈ 1/256 of positions, slightly less because a found gene skips 32 bits) and ~89 % of the DNA is junk. Shorter markers make random DNA dense with genes; longer markers make genes rare and junk dominant.

- `01111111` is **bifix-free**: no proper prefix equals a proper suffix (prefixes start with 0, suffixes are all 1s), so two marker occurrences can never overlap and the scan is unambiguous.
- A marker can also occur **inside** a payload. It is not read (the scanner jumps over the whole gene), but if a mutation destroys the outer marker, the hidden one can become active — a 'cryptic gene' that suddenly appears.

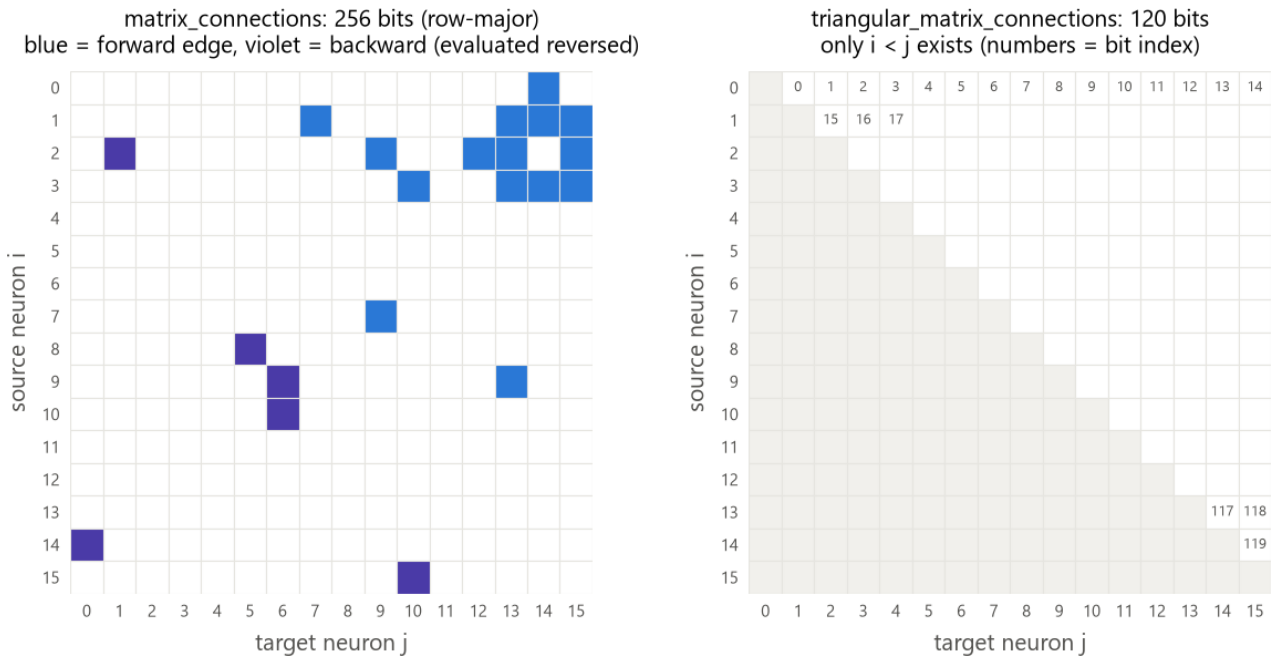
> **Caveat**
> `FastDNA.py` hard-codes `GENE_LEN = 32` (= 8-bit marker + 24-bit payload). The payload is read at `i + len(MARKER)`, but the jump after a gene, the end-of-genome bound and the number of bits copied by the crossover all assume 32. If you change `MARKER` to another length (e.g. a 6-bit marker), update GENE_LEN to `len(MARKER) + 24` as well.

8.3 connection_based — fixed 24-bit genes

From bit 32 on, every consecutive 24 bits are one gene: source byte, target byte, weight byte (same mappings as above). There is no junk and no marker: a genome of L bits always yields floor((L − 32) / 24) connections. Simple and dense, but every insertion or deletion shifts all later genes (see the table above).

8.4 matrix_connections — adjacency matrix + weight list

Bits 32–287 are a 16 × 16 adjacency matrix in row-major order: bit 32 + 16·i + j = 1 means a connection i → j. Connections are created in row-major order; then bytes from bit 288 on are assigned as their weights in that same order (connection k gets the k-th byte). Connections without a weight byte keep weight 0. Genomes shorter than 288 bits give an empty network.



Left: a matrix created by the matrix initializer (blue = forward edge, violet = backward edge, which the evaluator reverses). Right: the triangular variant only stores the 120 pairs with i < j.

- The 16 diagonal cells are self-loops and are ignored. Cells below the diagonal are backward edges: the evaluator reverses them, so cell (i, j) with i > j adds its weight to the connection j → i — both triangles feed the network.
- Low locality: switching one matrix bit on inserts a connection in the middle of the order, so every later connection receives the next connection's weight.

8.5 triangular_matrix_connections

Bits 32–151 store only the upper triangle (i < j): 15 + 14 + … + 1 = 120 bits, bit index $k = 15i - i(i-1)/2 + (j - i - 1)$. Every encoded connection is a forward edge, so nothing is wasted. Weights follow from bit 152 exactly as in the matrix decoder.

8.6 fixed_topology — weights only

The structure is fixed: every input (0–5) connects to every hidden neuron (6–11), and every hidden neuron to every output (12–15): 36 + 24 = 60 connections. The DNA only stores their weights, 8 bits each from bit 32 on (60 × 8 = 480 bits); missing bytes leave weight 0. This is the classic fixed-architecture neuro-evolution baseline.

8.7 grammar_matrix — an L-system that writes the matrix

Bits	Content
32–127	8 grammar rules × 12 bits: rule k rewrites symbol k (3 bits) into four 3-bit symbols
128–255	8 terminal rules × 16 bits: symbol k → 16 matrix bits
256 →	weight bytes, assigned to the connections in row-major order (as in matrix_connections)

Development: start with the symbol `000` → rule 0 gives 4 symbols → each of them is rewritten again → 16 symbols (two rewriting rounds). Each of the 16 symbols is then replaced by its 16-bit terminal rule and the 256 bits are read as the 16 × 16 matrix: **symbol k of the final string becomes row k** (the outgoing connections of neuron k). Example: if rule 0 = [3, 1, 1, 5] and rule 1 = [0, 0, 2, 2], the second round contains rule 1 twice, so rows 4–7 and 8–11 are identical. 224 structure bits describe a 256-bit matrix with strong regularity (repeated rows).

8.8 cellular_division — instruction list

From bit 32 on, the DNA is a program of 25-bit instructions. The cell list starts as `[neuron 0]`.

Opcode (1 bit)	Next 24 bits	Effect
0 = new cell	8 bits: id byte; 16 bits ignored	append a cell for neuron (byte % 16)
1 = connect	8 bits a, 8 bits b, 8 bits weight	connect cells[a % n] → cells[b % n] with the weight (n = current number of cells)

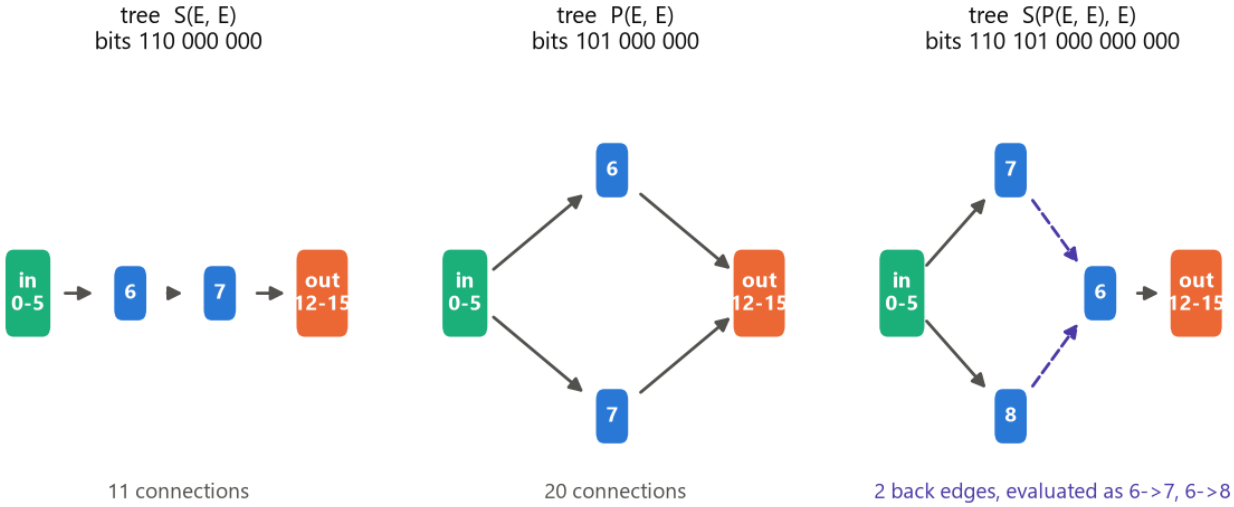
Order matters: a connect instruction can only refer to cells created before it, and the same cell index means different neurons depending on what was created earlier. The matching initializer first creates one cell per input and output neuron (cells 1–6 = inputs, 7–10 = outputs) and then adds 5–20 random input → output connections.

8.9 cellular_encoding — Gruau's developmental grammar

The genome is a **program tree** that a growing 'cell' executes, dividing into a network. Bits 32 onward are parsed depth-first, 3 bits per node; the bits after the tree are an 8-bit weight pool.

Code	Op	Arity	What the cell does
`000`	E	0	stop: become a neuron (threshold 0)
`001`	1	0	stop: become a neuron (threshold 1 — stored, not used by the evaluator)
`010 / 011`	M / W	1	wait one step, continue with the child (timing only)
`100`	R	1	recursion: life −1; if life > 0 restart at the root, else continue with the child
`101`	P	2	parallel division: two children, both inherit ALL input and output links
`110`	S	2	sequential division: child 1 takes the inputs, child 2 takes the outputs, new link child 1 → child 2
`111`	B	2	branch: left subtree if life > 1, else right subtree

Development: input and output neurons are created first and wired inputs → initial cell → outputs. Cells are processed first-in-first-out; when a cell terminates it receives the next hidden id (6, 7, … 11, then cycling). Finally every link between two neurons becomes a connection, weights taken from the pool in creation order (127 → −0.02 when the pool runs out). Limits: 500 cells, 5 000 steps, tree depth 200; `1` (initial life) = 1.



Three small trees decoded by the project's decoder ('in 0-5' stands for the 6 input neurons and 'out 12-15' for the 4 outputs).

Caveat
In the third tree the right child of S terminates **before** the children of P (FIFO order), so it gets id 6 while P's children get 7 and 8. The links 7 → 6 and 8 → 6 are backward edges; the evaluator reverses them into 6 → 7 and 6 → 8, which points them away from the outputs, so the inputs still do not reach the outputs (verified: all four outputs stay 0). Hidden-id assignment by termination order can cut paths that the tree describes.

8.10 Diploid decoders (Double_DNA_one_chromosome)

A diploid individual has two strands, DNAa and DNAb. There are two families of decoders:

- **Combine-based** (`connection_based`, `connection_based_markers`, `matrix_connections`, … , `cellular_encoding`): `dna.combine(method)` merges the two strands into one temporary strand, respecting the gene structure of that encoding, and the single-strand decoder decodes it. `combine` is random, so **the same individual can decode to a different network each time it is decoded** (every generation).
- **Dual** (`…_dual`): each strand is decoded on its own and the two connection lists are concatenated; connections present on both strands add up.

combine method	How the two strands are merged
random	one whole strand, 50/50
crossover	per bit 50/50
connection_based	24-bit blocks 50/50
connection_based_markers	marker-aware walk (older variant: if only one strand has a gene it is taken 50 % or skipped, and both positions jump 32)
matrix_connections / triangular_…	first 256 / 120 bits as one block, then 8-bit weight blocks
fixed_topology	8-bit blocks
grammar_matrix	12-bit grammar rules, 16-bit terminal rules, 8-bit weights
cellular_division	25-bit instruction blocks
cellular_encoding	two 16-bit rate chunks, then 3-bit tree-node chunks

> **Caveat**
> The diploid route is not runnable in the current engine: `decode_rates`, `bestDNA.txt` writing and the genotypic diversity read `dnaType.DNA`, which is `None` for a Double_DNA individual (verified: `decode_rates` raises TypeError). It needs a rates source for two strands (e.g. read them from DNAa) before it can be used.

9. Evaluating the network

9.1 connection_based_sort_feed_forward (Computation.py)

```
values = {input_id: sensor value}                       # everything else starts at 0
forward = [(s, t, w) if s < t else (t, s, w)            # backward edges reversed,
           for each connection with s != t]             # self-loops dropped
sort forward by target id
for each target t (in increasing order):
    values[t] += sum(values[s] * w for its incoming edges)
    if t is not an input: values[t] = tanh(values[t])   # inputs stay linear
return [values[12], values[13], values[14], values[15]]
```

Because every edge goes from a lower to a higher id, processing targets in increasing order guarantees that all sources are final before they are used: one pass evaluates the whole network. A neuron without incoming edges outputs 0 (tanh(0) = 0 for hidden/outputs). Consequence: a network with no path to output 14 (accelerate) never moves and dies 'stalled' after 10 s.

9.2 The dense equivalent used for training

`build_weight_tensors` turns every network into a 16 × 16 matrix W with W[s, t] = sum of weights of all s → t connections with s < t plus all t → s connections (reversed backward edges); self-loops are skipped. NumbaSim then computes, for t = 1 … 15: `acc = Σ_{s<t} val[s]·W[s,t]`; inputs: `val[t] += acc`; others: `val[t] = tanh(acc)`. This is the same computation as Computation.py, except for floating-point summation order (checked 2026-10-10: max difference 1.3e-15 over 500 random networks containing ~4 900 backward edges).

9.3 connection_based_max_range_propagation (recurrent, not active)

Uses **all** edges, including self-loops and backward edges. Starting from zeros, it updates every non-input neuron synchronously (`tanh` of the weighted sum of the previous values) up to 10 times or until nothing changes by more than 1e-4. There is no memory between frames (each call starts from zero). Only Car.py can use it; the headless simulation does not implement it (see the caveat in §4).

10. Initialisation operators

Initializer (single DNA)	What it creates	Length
random_bits (ACTIVE)	32 random rate bits + uniformly random body; numpy uint8 array	32 + 5 000…10 000
connection_based	5–20 'good' genes, each a direct input → output connection with a random weight, followed by 24 junk bits (which the fixed-frame decoder reads as another random gene)	32 + 48 per good gene
connection_based_markers	5–20 × [MARKER, good input→output gene, MARKER, 24 junk bits]. The second MARKER + 24 junk bits is itself a gene, so every good gene is followed by a random one (measured: 800 bits → 24 genes)	32 + 64 per good gene
matrix_connections	256-bit matrix with 5–20 input→output bits and 5–20 random bits set, then random weight bytes	32 + 256 + 16·n
triangular_matrix_connections	same idea on the 120-bit triangle	32 + 120 + 16·n
fixed_topology	60 random weight bytes	512
grammar_matrix	96 + 128 random rule bits + 400–800 random weight bits	656–1 056
cellular_division	one 'new cell' instruction per input and output neuron, then 5–20 input→output connect instructions	32 + 25 per instruction

The diploid initializers call the single one twice (one call per strand). There is no dedicated initializer for cellular_encoding (random_bits works: any bit string is a valid tree). The structured initializers give generation 0 a head start of plausible input→output wiring; `random_bits` (active) does not — everything has to be discovered.

Two other ways individuals are created: `LOAD_MODEL = True` (engine attribute) loads `assets/bestDNA.txt` into the first individual of island 0; `RESUME_CHECKPOINT` reloads a whole population (§15).

11. Selection

11.1 tournament_selection (active)

Draw 3 individuals uniformly **with replacement** from the island, return the fittest (a reference, not a copy). Each child needs two independent tournaments, so both parents can be the same individual.

Rank in island of 100	P(win one tournament)	Expected times chosen as a parent (200 draws)
1	0.0297	5.94
2	0.0291	5.82
10	0.0246	4.91
25	0.0171	3.42
50	0.0077	1.53
75	0.0020	0.39
100	0.0000	0.00

P(rank r wins) = $((N - r + 1)^3 - (N - r)^3) / N^3$. The best individual is chosen ~3× as often as under uniform choice, the median ~0.8×, the worst essentially never. Ties are broken by draw order. The pressure is moderate and does not depend on how large fitness differences are (only ranks matter), which keeps the huge finish bonus from taking over.

11.2 roulette_selection

Fitness-proportional: P(i) = f_i / Σf. If all fitnesses are 0 it returns the first individual. With this project's fitness scale it is very aggressive: one finisher with 400 among cars with ~10 takes most draws.

12. Crossover

12.1 Common frame of every single-DNA crossover

```
child = parent1.clone()
p = (parent1.crossover_rate + parent2.crossover_rate) / 2
if random() < p:
    child.DNA = <operator>(parent1.DNA, parent2.DNA)
child.crossover_rate, child.mutation_rate = decode_rates(child.DNA)   # child's own rates
```

Without crossover the child is a copy of parent 1 (and is then mutated). The child's mutation rate — used immediately by the mutation step — comes from the child's own (possibly mixed) rate bits.

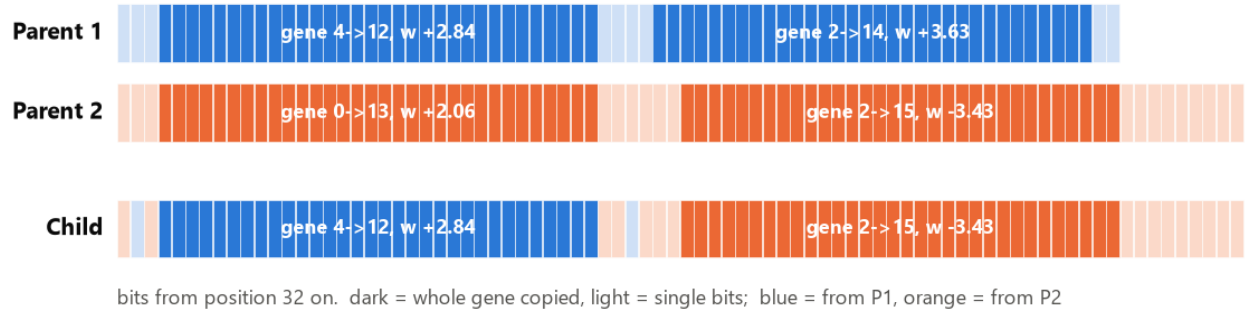
12.2 connection_based_markers — the active crossover

Implemented in `FastDNA._crossover_markers` (Numba). One index i walks along **both** parents at the same time:

```
bits 0..31: each bit from P1 or P2 with probability 1/2
i = 32
while i < len(P1) and i < len(P2):
    g1 = gene starts at P1[i]?   g2 = gene starts at P2[i]?
    neither:  take P1[i] or P2[i] (1/2 each);                    i += 1
    only one: 1/2 -> copy that whole 32-bit gene;                i += 32
              1/2 -> take one bit, P1[i] or P2[i] (1/2 each);    i += 1
    both:     copy P1's or P2's 32-bit gene (1/2 each);          i += 32
tail: 1/2 -> rest of P1 from i, else rest of P2 from i
```

Traced example

Two small parents (genome lengths 105 and 114), crossed with a fixed random seed by a Python mirror of the Numba function. The mirror was checked to produce bit-identical children to `FastDNA._crossover_markers` on 300 random parent pairs.



Parents and the child, from bit 32. The child keeps P1's first gene, takes a few single bits, then P2's second gene; P1's second gene is lost.

Position i	Gene at P1[i] / P2[i]	Decision	Bits added
0-31	—	per-bit 50/50	32
32	no / no	bit from P2	1
33	no / no	bit from P1	1
34	no / no	bit from P2	1
35	yes / yes	copy P1 gene	32
67	no / no	bit from P2	1
68	no / no	bit from P2	1
69	no / no	bit from P1	1
70	no / no	bit from P2	1
71	yes / no	no gene; bit from P2	1
72	no / no	bit from P2	1
73	no / yes	copy P2 gene, jump 32	32
105	—	rest of P2	9

- At 35 both parents start a gene: one of them (P1's 4→12) is copied whole and both sides jump 32 bits.
- At 71 only P1 has a gene; the coin chose 'single bit', so P1's gene 2→14 is not copied. At 73 P2's gene 2→15 starts; it is copied and i jumps to 105 — the rest of P1's gene under it is skipped as well.
- P1 ends at 105, so the loop stops; the coin picked P2's tail (9 bits). The child has P2's length (114).

Properties

- **Unbiased:** every decision is 50/50; fitness is not used (selection already used it once).
- **Genes are never cut in the middle** by the crossover: a gene is either copied whole or (in the 'single bit' branch) its first bit is taken and the scan continues, so it is lost unless the other parent's bits rebuild a marker.
- **Alignment is positional:** genes compete with whatever starts at the same index in the other parent, not with a matching gene. A gene that exists in only one parent survives with probability 1/2 at its position.
- The child's length is about the shorter parent's length plus the tail of a random parent.

12.3 Simple cut crossovers (legacy)

Shown on P1 = `111111111111`, P2 = `000000000000` (12 bits):

Method	What it does	Example child
single_cut	cut in [0, min_len − 1]; P1[:cut] + P2[cut:]	cut 5 → 11111 0000000

Method	What it does	Example child
multi_cut	two cuts c1 < c2; P1[:c1] + P2[c1:c2] + P1[c2:]	3, 8 → 111 00000 1111
random_bit	uniform: every bit 50/50 up to the shorter length, then the longer parent's tail	e.g. 101100101101
single_cut_weighted	cut ~ Triangular(0, L, mode = w1·L), w1 = f1 / (f1 + f2)	f1 = 300, f2 = 100 → mode at 75 %, child mostly P1
multi_cut_weighted	segment length ~ Triangular(0, L, mode = w2·L); that segment of P2 replaces P1's at a random place	f2 small → short P2 segment
random_bit_weighted	every bit from P1 with probability w1; tail from the fitter parent	f1 = 300, f2 = 100 → 75 % P1 bits
no_crossover	child = copy of parent 1	111111111111

The weighted variants apply fitness twice (once in selection, once in crossover), which increases selection pressure and works against diversity.

12.4 Block-aligned crossovers for the other encodings (legacy)

All of these mix the 32 rate bits per bit (with probability w1 for parent 1), then swap whole structural units with probability w1, and take the remaining tail from the fitter parent:

Method	Unit swapped	Matches decoder
connection_based	24-bit genes aligned at 32 + 24k	connection_based
matrix_connections	the whole 256-bit matrix, then 8-bit weights	matrix_connections
triangular_matrix_connections	the whole 120-bit triangle, then 8-bit weights	triangular
fixed_topology	8-bit weights	fixed_topology
grammar_matrix	each 12-bit grammar rule, each 16-bit terminal rule, 8-bit weights	grammar_matrix
cellular_division	25-bit instructions	cellular_division

12.5 Diploid crossovers (legacy)

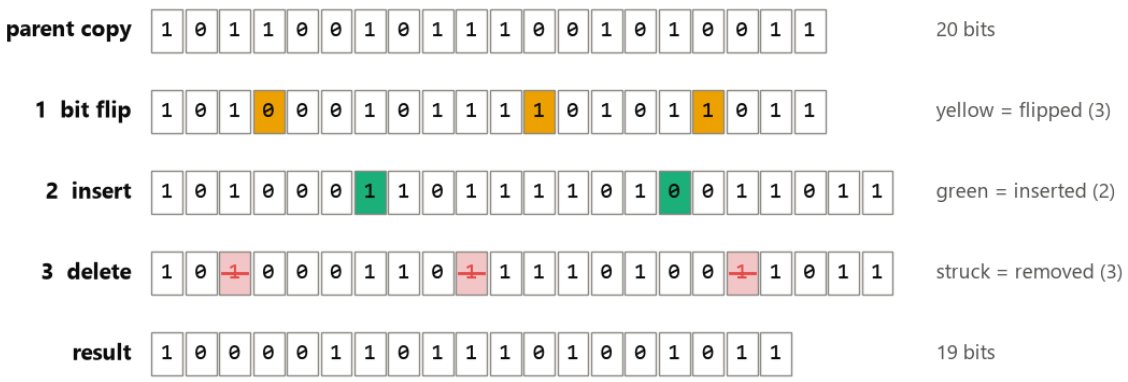
Each parent produces one 'gamete' from its two strands with `combine(method)`; the child's DNAa is parent 1's gamete and DNAb parent 2's (like meiosis). `random_strand_pick` passes one whole strand from each parent. The rates are not re-decoded afterwards.

13. Mutation

13.1 The active chain

`MutationCombiner([flip, insert, delete]).mutate` applies the three operators in that order, each with the child's own `mutation_rate` p. All three are NumPy-vectorised (FastDNA), return a new array, and treat the 32 rate bits like any other bits.

Operator	Per bit	Effect on length
random_bit_flip	with probability p: 0 ↔ 1	none
random_insert	with probability p: insert one random bit right after it	× (1 + p)
random_delete	with probability p: remove it	× (1 − p)



The three steps on a 20-bit toy string (positions chosen by hand for illustration).

13.2 What a given mutation rate means

mutation rate p	Flips / inserts / deletes in a 7 500-bit genome	Expected length change	P(a 32-bit gene survives untouched)
0.5	3,750.0 each	× 0.7500	2.52e-29
0.1	750.0 each	× 0.9900	4.5e-05
0.01	75.0 each	× 0.9999	0.385
0.001	7.5 each	× 1.0000	0.909
0.0001	0.8 each	× 1.0000	0.991

A gene is untouched if none of its 32 bits flips, none of the 31 slots inside it receives an insertion and none of its 32 bits is deleted: ≈ (1 − p)^95. At the initial average p = 0.5 every gene of every child is destroyed, so selection immediately favours genomes whose rate bits happen to be low — that is the collapse visible in the first 50–100 generations. Insert and delete at the same rate keep the expected length almost constant ($(1 + p)(1 − p) = 1 − p^2$), but the length performs a random walk, and lineages with long genomes carry more junk.

13.3 Structure-aware mutations (legacy)

These fire **once per individual** with probability = mutation_rate (not per bit) and change whole units:

Method	Operations (one chosen at random)
connection_based	duplicate a 24-bit gene into a random gene slot, or delete one (if more than one)
connection_based_markers	find all marker genes; duplicate one to a random position ≥ 32, or delete one (if more than one)

Method	Operations (one chosen at random)
matrix_connections / triangular	duplicate or delete an 8-bit weight block in the weight tail
fixed_topology	copy one weight over another, or delete a weight block (only if the tail is longer than 128 bits)
grammar_matrix	swap two grammar rules, copy a rule over another, the same for terminal rules, or duplicate/delete a weight
cellular_division	duplicate or delete a 25-bit instruction

They use `deepcopy` and Python lists (slow), and `connection_based` fails on numpy DNA (§19). Diploid mutations: bit flips per strand; insert/delete at the same positions in both strands so the strands stay equally long; structure-aware ones apply the single-strand version to each strand.

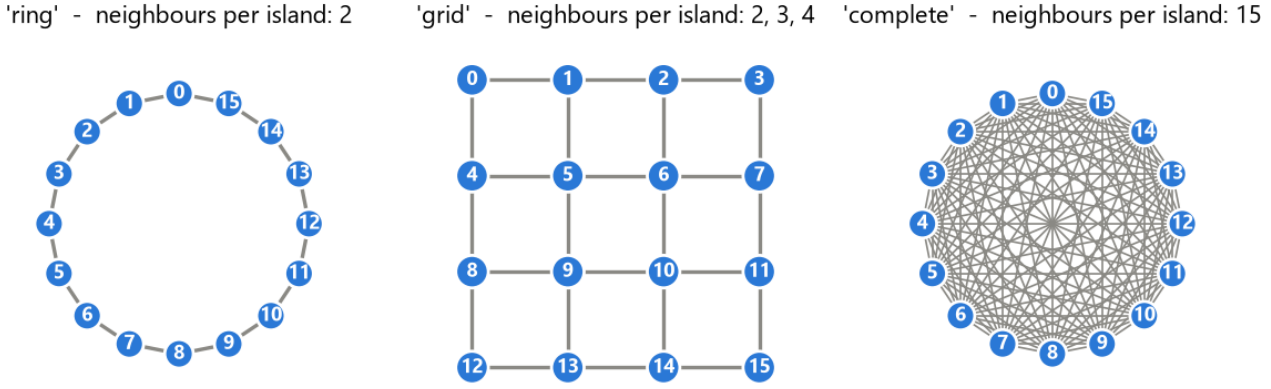
14. Population structure and the generation loop

14.1 The loop, step by step (runEvolutionIslands)

1. **Advance**: `islands = [list(isl) for isl in new_islands]` (shallow copies).
2. **Decode** every individual: `nn = DNA_decode(dnaType)`, `crossover_rate, mutation_rate = decode_rates(dnaType)`.
3. **Evaluate**: all islands are concatenated into one flat population and `evaluate_fitness()` runs once (one Numba call for all new individuals); fitnesses are written back per island.
4. **Sort** each island by fitness, best first.
5. **Validate** the global best (max of the island leaders) on the validation track → `best_indv_val_fitness`.
6. Mark everybody `new = False` (so carried-over individuals keep their cached fitness).
7. **Migration plan** if `generationNumber % MIGRATION_INTERVAL == 0`.
8. **Breed** every island (`evolve_island`).
9. Print, `generationNumber += 1`, save results, checkpoint, maybe draw graphs.

14.2 Island topologies

`SUBPOP_SIZE = POPULATION_SIZE // NUM_SUBPOPULATIONS` (the remainder is dropped). `_build_island_graph` turns `ISLAND_GRAPH` into neighbour lists:



The three built-in topologies for 16 islands. 'grid' (active) uses cols = floor(√n), rows = ceil(n / cols), 4-neighbourhood, no wrap-around: corners have 2 neighbours, edges 3, inner islands 4.

- `'ring'`: i ↔ i ± 1 (wrapping). Slowest spread of good solutions, most isolation.
- `'complete'`: everyone is a neighbour of everyone. Fastest mixing — closest to a single population.

- `'grid'`: in between. Your parameters.txt records the 16-island grid as `[[4, 1], [5, 0, 2], [6, 1, 3], [7, 2], [0, 8, 5], …]`.
- A list of lists of 0/1 (adjacency matrix, n × n) or a ready adjacency list is also accepted. Any other string falls back to a ring.

14.3 Migration (_plan_migration)

On a migration generation, every island takes its K = min(MIGRATION_SIZE, size − 1) best individuals. Each of them is cloned, reset (`new = True`, fitness 0, no network) and sent to **one randomly chosen neighbour** — each migrant chooses independently. So MIGRATION_SIZE is per island, not per edge (the comment in main.py says 'along each edge'). An inner grid island can therefore receive 0 to 4 immigrants in one migration.

- **Move semantics:** an emigrant is not also kept at home as an elite (elites are taken from the ranks after the emigrants). Its genes can still be inherited at home, because parents are selected from the full evaluated island.
- With `ELITE_FRACTION = 0` (current), migrants are the **only** individuals that enter a new generation unchanged (no crossover, no mutation): every 50 generations, each island's leader survives intact — on a neighbour.

14.4 Building the next generation of an island (evolve_island)

Fill order	Who	Count (current config, subpop 100)
1 elites	best non-emigrants: ranks n_emigrants … n_emigrants + elite_amount − 1, cloned, fitness cached	int(100 × ELITE_FRACTION) = 0
2 immigrants	migrants that arrived from neighbours	0 (most generations) or 0–4
3 random	fresh DNA_init() individuals	int(100 × 0.0) = 0 (hard-coded 0 % in island mode)
4 children	selection ×2 → crossover → mutation, until the island is full	100 minus the above

14.5 Single-population mode

With `NUM_SUBPOPULATIONS = 1`, `run()` calls `runEvolution` instead — a different, older algorithm: deep copies every generation, `elite_amount = POPULATION_SIZE × ELITE_FRACTION` elites, **10 % fresh random individuals every generation** (`new_amount = 0.1 × POPULATION_SIZE`), the rest children; validation of the best; the `graph()` plot variant. So setting 1 island changes more than the island count.

14.6 Fitness caching: the `new` flag

Children, immigrants and random individuals have `new = True` and are simulated. After evaluation everybody is set to `new = False`; an elite cloned into the next generation keeps `new = False` and its fitness, so neither route re-simulates it (the track is fixed, so the result would be identical). Validation caches `testFitness` the same way.

14.7 Randomness and reproducibility

- `random` (seeded): DNA init (`generateRandomDna`, lengths), lap-2 shuffles, tournament draws, crossover coin flips (Python operators), migration targets, Jaccard subsampling.
- `numpy.random` (seeded): mutations; it also seeds Numba's own PRNG once, which the compiled crossover uses.
- The simulation itself is deterministic. Same seed + same code + same window size → same run.

15. Saving, checkpoints and resuming

15.1 Checkpoint files

Every `CHECKPOINT_INTERVAL` generations (100) the evaluated population of the just-finished generation is written to `<run>/checkpoints/gen_000100.txt` etc. (~9 MB at 1 600 individuals):

```
GENERATION 100
NUM_SUBPOPULATIONS 16
POPULATION_SIZE 1600
ISLAND 0 100
00110000101011000000000000011000011100000000011011100...   <- one line per individual
...
ISLAND 1 100
...
```

15.2 Resuming a run

1. Set `RESUME_CHECKPOINT = "evolutionOutput/<run>/checkpoints/gen_000500.txt"` in main.py (keep the same island count).
2. Start with a seed as usual. The islands are rebuilt from the file (all individuals `new = True`, fitness 0) and `generationNumber` is set to 500, so generation 500 is evaluated once more and evolution continues.

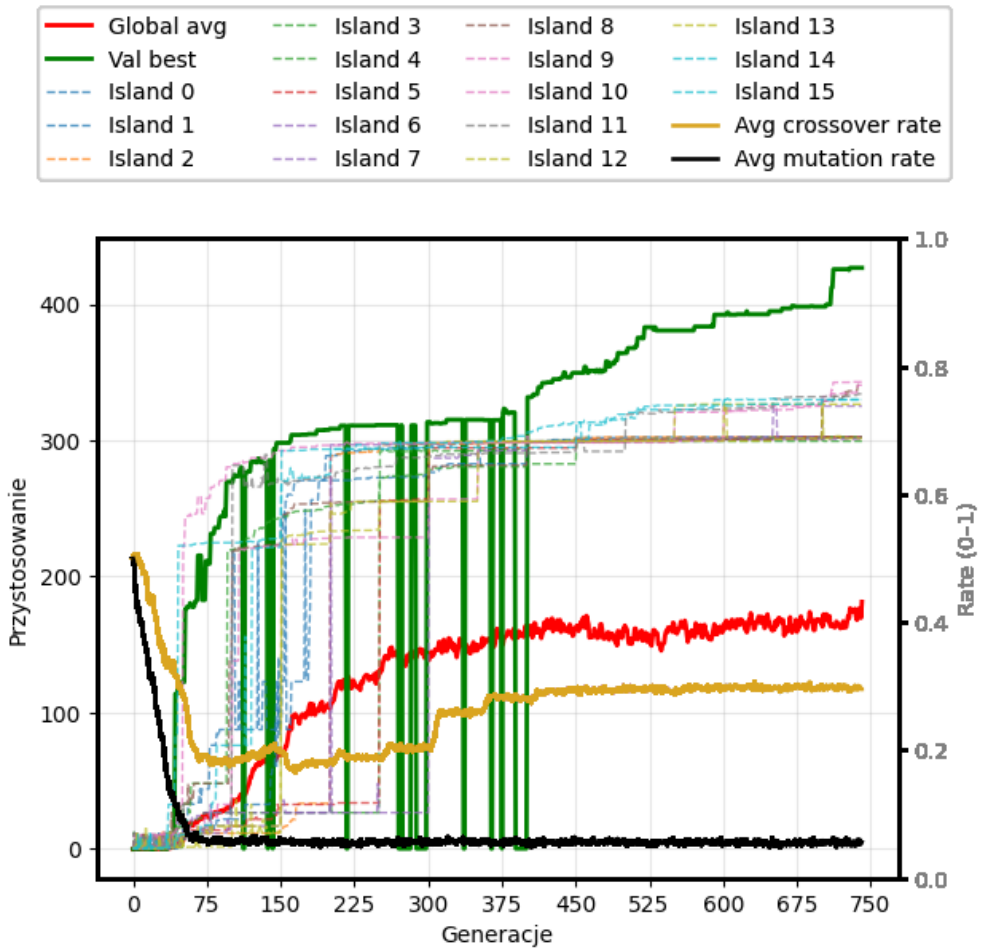
> **Note**
> A resumed run writes into a **new** output folder and its graphs start from an empty history (row 0 = the resumed generation); the console generation counter continues from the checkpoint. Checkpoint lines are loaded as Python lists; the first crossover/mutation turns them back into numpy arrays.

15.3 bestDNA.txt and bestValDNA.txt

- `bestDNA.txt` is overwritten every generation with the current global best (by train fitness). Because there are no elites, it can be worse than an earlier generation's best.
- `bestValDNA.txt` is written only when the validated individual sets a new validation record. This is the file to use as 'the' result of a run (test.py also reads it).
- Both are plain bit strings without line breaks; load with `[int(c) for c in open(path).read().strip()]`.

16. Graphs and console output explained

16.1 bestFitnessGraph.png



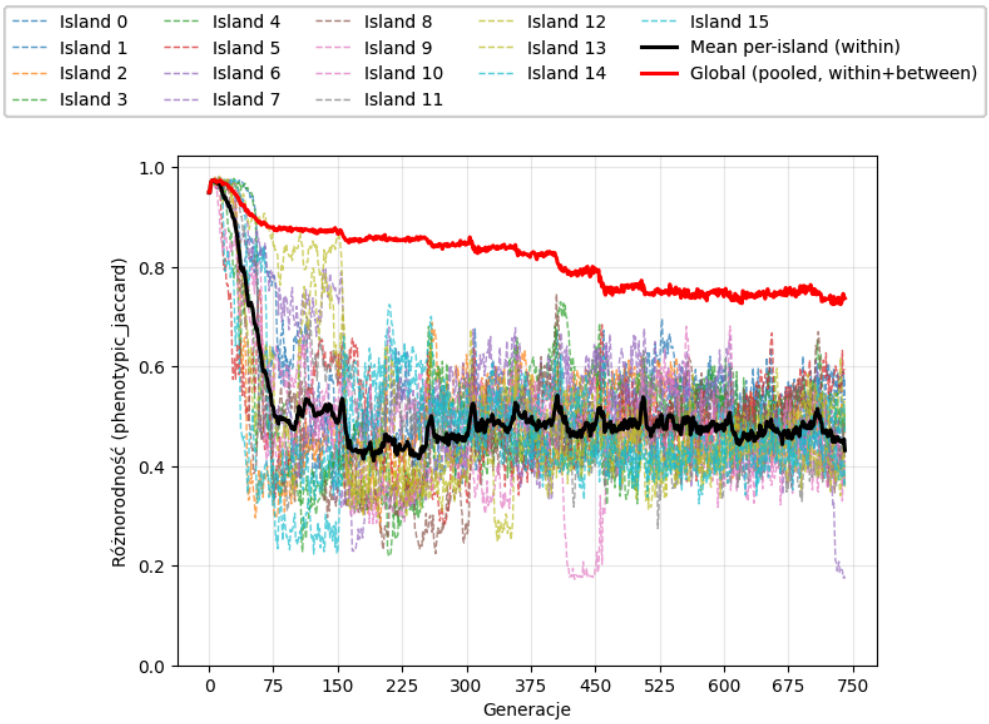
Your run new_train_val_tracks_success after 741 generations (island mode).

Line	Meaning
Global avg (red)	mean train fitness of all 1 600 individuals (the code stores it in `medianFitnessList`, but it is the mean)
Val best (green)	validation fitness of **this generation's** global-best individual — not the best ever. It drops to ~0 whenever the current train leader cannot drive the validation track
Island k (dashed)	best train fitness inside island k; the upper envelope of all dashed lines is the global best
Avg crossover rate (gold, right axis 0–1)	mean decoded crossover rate
Avg mutation rate (black, right axis 0–1)	mean decoded mutation rate — dominated by outliers, median ≈ 0.0001 (§7.2)

- Generations 0–45: nothing works and the mutation rate falls from 0.5. Island 14 is the first to finish the 20-screen train track (fitness > 200) at generation 46, island 10 at 51.
- Most other islands first exceed 200 at generations **101, 151, 201 and 251 — one generation after a migration** (migrations happen at 100, 150, …). They did not find a finishing driver themselves; they received one from a neighbour. This is migration visibly spreading a solution through the 4×4 grid.
- Flat steps shared by several islands at exactly the same value (302.33) are the **same network** spread by migration and kept alive by faithful copying (low mutation rates).
- The green line climbs to 426 by generation 740: later leaders also finish the unseen validation track, faster.

- In single-population mode the graph shows Best fitness (train, blue), Avg fitness (train, red), Best fitness (val, green) and the two rates.

16.2 diversityGraph.png



Phenotypic diversity of the same run.

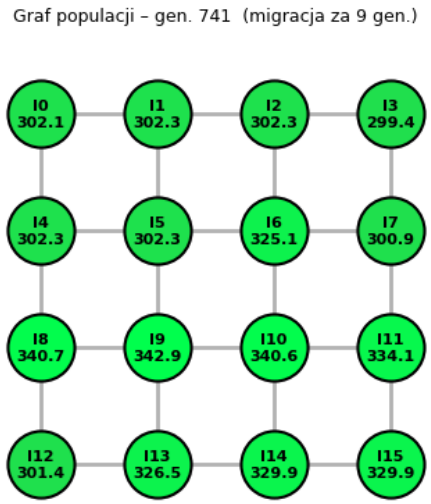
Default metric `phenotypic_jaccard`: for two networks with edge sets A and B (the edges as the network evaluates them: a backward edge counts as its reversed forward edge, self-loops are dropped, weights are ignored) the distance is 1 − (shared edges) / (edges in A or B); the value is the mean over all pairs. Example: A = {2→14, 2→8, 8→12}, B = {2→14, 0→13} → 1 − 1/4 = 0.75. 0 = all networks share one topology, 1 = no shared edges.

Line	Meaning
Island k (dashed)	diversity inside island k
Mean per-island (black)	average within-island diversity
Global (red)	diversity of the pooled population (random 600 individuals): within + between islands. The gap to the black line is **between-island** diversity

In this run within-island diversity falls from 0.97 to ~0.45 by generation 75 and stays there, while global diversity stays at 0.75–0.85: the islands hold **different** solutions — the island model is doing its job against premature convergence. Only the per-island values are saved to `diversityData.txt`; the global curve exists only in the image. (This run used the older signature with raw (source, target) pairs; since 2026-10-10 genes 9 → 2 and 2 → 9 count as one edge, so new diversity values are not directly comparable with older graphs.)

DIVERSITY_METRIC	Definition	Range
phenotypic_jaccard (default)	mean pairwise Jaccard distance of edge sets	0 … 1
genotypic	mean over bit positions of 2p(1 − p), p = share of 1s at that position. With variable-length marker genomes positions are not aligned, so it mostly measures noise	0 … 0.5
unique_fraction	number of distinct topologies / island size	1/n … 1
fitness	std / mean of fitness (coefficient of variation)	≥ 0

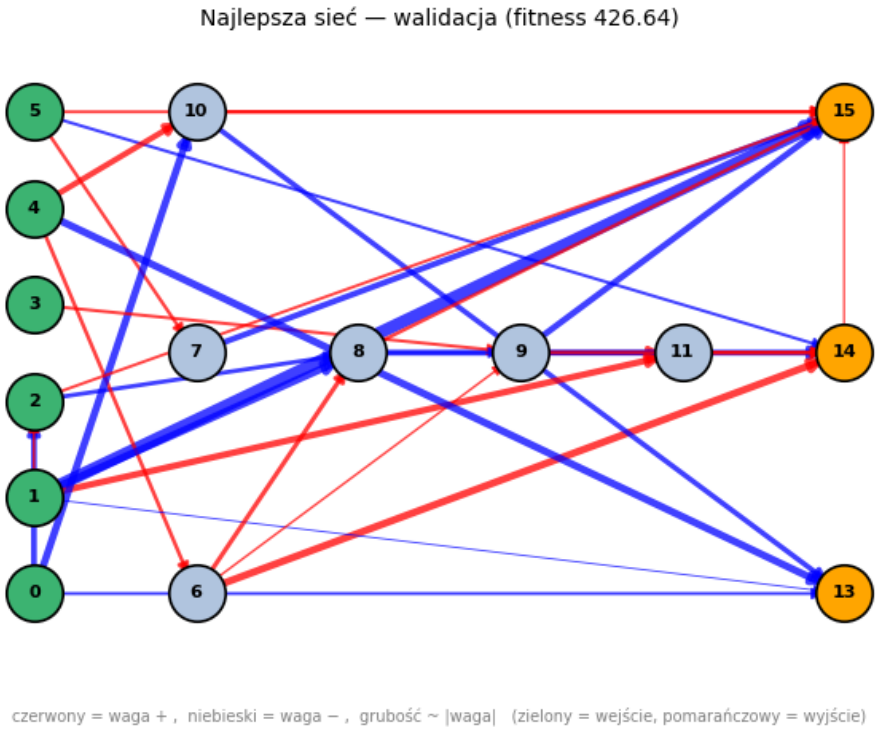
16.3 islandTopology.png



Island graph at generation 741.

One node per island, laid out on the grid (island i at row, col = divmod(i, cols)); grey lines are the neighbour links. Label: I{k} and the island's best train fitness. Colour goes from red (low) to green (the best island): RGB = (1 − r, r, 0.3) with r = island best / best island. Title: generation and generations until the next migration (*Graf populacji – gen. 741 (migracja za 9 gen.)*). The grid layout is used for every topology.

16.4 bestTrainNetwork.png and bestValNetwork.png



The network that set the validation record (fitness 426.64) in that run, drawn by the code of that time (backward edges were still discarded then).

- **Train** = the best network ever seen on the train track (kept even after it disappears from the population). **Val** = the network of the individual that set the latest validation record. In this run both are the same network.
- Only edges that matter are drawn: backward edges reversed and self-loops dropped (as in the evaluator), duplicates summed, and only edges on some input → output path.

- Red = positive weight, blue = negative, thickness ~ |weight|. Green = inputs, orange = outputs, grey-blue = hidden. Columns = longest path from the inputs; outputs right-aligned.
- Reading this one (as evaluated when the run was made): 31 of its 73 genes lie on input → output paths. The strongest single weight is 1 → 15 = −9.06 (two duplicate genes summed): the more open the 25°-right ray, the less it brakes. 4 → 13 = −4.92: an open 50°-left ray suppresses turning right. Speed feeds brake positively (5 → 15 = +1.35) and accelerate negatively (5 → 14 = −1.43) — a built-in speed limiter. Most behaviour, however, runs through hidden neurons 6–11.
- If no input → output path exists the image says *brak ścieżki wejście → wyjście* (no input → output path).

16.5 Console output per generation

```
  >> Migracja: przeniesiono 16 osobników                (migration generations only)
Generacja nr. 100
Nowy najlepszy wynik walidacyjny: 269.100 -> zapisano checkpoint   (only on a new val record)
Globalny najlepszy: 281.600  |  Walidacja: 269.100  |  Średnia: 31.500
Avg crossover_rate: 0.1810  Avg mutation_rate: 0.058900
  Wyspa 0 (sąsiedzi: [4, 1]): 25.312
  ...
Różnorodność (phenotypic_jaccard) globalna: 0.8800  |  śr. wyspy: 0.5000  (między-wyspowa: 0.3800)
  >> Zapisano checkpoint populacji: evolutionOutput/.../gen_000100.txt (1600 osobników)
```

Translations: Migracja: przeniesiono N osobników = migration: N individuals moved; Generacja nr. = generation no.; Nowy najlepszy wynik walidacyjny … zapisano checkpoint = new best validation score … saved; Globalny najlepszy = global best; Walidacja = validation; Średnia = mean; Wyspa (sąsiedzi) = island (neighbours); Różnorodność globalna / śr. wyspy / między-wyspowa = diversity global / island mean / between islands. (Format example; the fitness and rate numbers are generation 100 of your run, rounded; island and diversity numbers are placeholders.)

17. Parameter reference

17.1 main.py

Parameter	Now	What it changes / how
seed (argv[1])	required	Seeds random and numpy. Changes the initial population, lap-2 shuffles and every random decision. Run several seeds per experiment — results vary a lot between seeds.
MAX_GENERATIONS	10000	Length of the run. The last generation is always graphed.
POPULATION_SIZE	1600	Total over all islands; each island gets POPULATION_SIZE // NUM_SUBPOPULATIONS (remainder dropped). Time per generation grows ~linearly.
ELITE_FRACTION	0	Share of each island copied unchanged (best non-emigrants, fitness cached). 0 = no elitism (current research choice). Example: 0.02 with islands of 100 → 2 elites per island. Higher = faster but greedier convergence.
DRAW_TRAIN_TRACK / DRAW_VAL_TRACK	False	True = draw segments interactively at start (overwrites 1.png, 2.png, … — §5.4); False = load the folders.
NUM_SUBPOPULATIONS	16	Number of islands. 1 = switches to the single-population algorithm (§14.5). More islands = more isolation and between-island diversity, smaller islands = more drift.
MIGRATION_INTERVAL	50	Migration happens on generations divisible by it. Smaller = islands share solutions sooner (less diversity); larger = more independent evolution.
MIGRATION_SIZE	1	Best K of each island emigrate, each to a random neighbour (capped at island size − 1).
ISLAND_GRAPH	'grid'	'ring' | 'complete' | 'grid' | adjacency matrix (n×n of 0/1) | adjacency list. Controls how fast solutions spread (§14.2).
CHECKPOINT_INTERVAL	100	Full-population dump every N generations; 0 disables. ~9 MB per file at 1 600 individuals.
RESUME_CHECKPOINT	None	Path of a checkpoint to continue from (§15.2).

Parameter	Now	What it changes / how
SENSOR_STEP	1	Ray-march step in px for every route (Numba training sim, NumPy fallback, Car.py validation and drawing). Larger = faster sensing but readings quantised to the step, and a ray can step over walls thinner than the step. Written to parameters.txt.
INPUTS	[0..5]	Neuron ids for the 5 rays (right 50° … left 50°) and speed. Must have exactly 6 entries (the simulation hard-codes 5 rays + speed). Use the lowest ids so they can feed everything.
OUTPUTS	[12..15]	Ids for turn-left, turn-right, accelerate, brake; must have 4 entries. NR_OF_NEURONS = last + 1, so this also sets the number of hidden neurons: [28..31] → 32 neurons with 22 hidden (ids become byte % 32). Keep outputs the highest ids, otherwise hidden neurons cannot feed them.
MARKER	01111111	Gene start pattern for the marker decoder/crossover/initializer. Length sets gene density (§8.2). Keep it bifix-free; keep 8 bits unless you also update FastDNA.GENE_LEN.
DNA_INITIALIZATION … MUTATION	see §1	The operator callables. Pick matching families (initializer, decoder, crossover, structure mutation of the same encoding).

17.2 Constants inside EvolutionEngine.__init__

Name	Value	Meaning
FPS	120000	pygame clock cap; effectively 'as fast as possible' (only matters when drawing)
COLLISION_SURFACE_COLOR	GREEN (0, 255, 0)	pixel colour treated as wall
LOAD_MODEL	False	True = put assets/bestDNA.txt into the first individual
DRAW_SIMULATION	False	route switch (§4)
WINDOW_WIDTH / HEIGHT	90 % of the display	also the size of one segment; 1536 × 960 here
CAR_WIDTH / CAR_HEIGHT	0.008 W / 0.6 × width	car body for collision and sprites
MIN_BRUSH_SIZE	round(1.1 × CAR_HEIGHT)	minimum brush radius in drawing mode
SEGMENT_WIDTH	WINDOW_WIDTH	unit of fitness (10 points) and of sensor normalisation
MAX_SENSOR_DIST	1.0 × SEGMENT_WIDTH	ray cap of the NumPy fallback only
SENSOR_STEP	from main.py (1)	passed to simulate_numba, simulate and every Car
GRAPH_INTERVAL	3	graphs rendered every N generations (data still logged every generation)
DIVERSITY_METRIC	'phenotypic_jaccard'	see §16.2
new_amount (islands)	0 %	random fresh individuals per island per generation (hard-coded in evolve_island)
new_amount (single pop.)	10 %	same, in runEvolution
frame cap	400000	argument of simulate_numba in evaluate_fitness

17.3 Car physics constants (keep in sync in three places)

The same constants are written separately in `Car.py`, `NumbaSim.py` (inside `simulate_numba`) and `BatchedSim.py` (module top). Changing one without the others makes training and validation disagree. The exception is SENSOR_STEP, which now comes from main.py for all three.

Constant	Car.py	NumbaSim	BatchedSim	Effect of increasing it
MAX_SPEED	4	4	4	faster cars, harder corners, faster finish bonus
ACCELERATION	2	2	2	throttle/brake change speed faster per frame
TURN_SPEED (°/frame)	3	3	3	tighter turning radius
SENSOR_ANGLES (°)	25	25	25	rays spread wider (±2 × value)
SENSOR_STEP (px)	main.py	main.py	main.py	one parameter (default 1): coarser, faster ray marching

Constant	Car.py	NumbaSim	BatchedSim	Effect of increasing it
speed input (in5)	speed / MAX_SPEED	same	same	— (normalisation to 0–1)
MIN_FITNESS_PROGRESS	0.1	0.1	0.1	more new progress needed to reset the no-progress timer
no-progress window	12000	12000	12000	frames allowed without progress
stall window	1200	1200	1200	frames allowed standing still
finish bonus	50000/(s+1)	same	same	weight of speed vs. distance

18. Legacy and auxiliary files

18.1 test.py

A post-hoc evaluation script from the circular-track era: for a parent folder with exactly 4 seed folders it loads each `bestValDNA.txt`, drives it on `assets/test_track1..3.png` from start positions in `assets/parameters.txt` (CAR_TEST_X1 …), and reports per track: pace (generation when train/val fitness first reached 100), success rates, generalisation (mean score), stability (std of scores), quality (speed variance of finishers) and an overtraining ratio (generation of the last val record / total). Results go to `test_results.txt/.csv`; `combine_results` merges experiments.

> **Caveat**
> test.py does not work with the current code: it constructs `Car(start, angle, surface, nn, …)` with the old argument order (the current Car has no surface argument and needs track width/height), it keeps only graphData rows with exactly 4 columns (current files have 6 + one per island, so every row is skipped), its thresholds assume the old lap-based fitness, and the test tracks are circuits rather than left-to-right segments.
>
> A new test script for the linear track would assemble a third set of segments (e.g. `assets/test/`) and use `NumbaSim` or `Car` with the current signature.

18.2 ideas.py

Not imported anywhere. Contains earlier prototypes: a Gray-coded marker decoder (marker `11110000`), point/indel mutations and gene duplication, a 'homologous' cut-at-marker crossover, a diploid organism that averages weights of connections present on both chromosomes, and several network runners (multi-step propagation, target-sorted feed-forward — the idea that became `connection_based_sort_feed_forward`).

18.3 scetch.txt, the CSV and old assets

- `scetch.txt` (Polish): the original GA design (elites, random injection, tournament, crossover/mutation probabilities), the planned experiment series (7 decodings × init / mutation / crossover variants, feed-forward vs recurrent, population size, diploid DNA), the grammar idea and notes such as 'many more seeds are needed because training is unstable'.
- `dna_marker_simulation_extended.csv`: gene statistics for markers of length 2–8 (table in §8.2).
- `assets/old/`, `assets/old_segs/`: older circular tracks and earlier segment sets; nothing loads them.
- `evolutionOutput/`: one folder per run (timestamps, or folders you renamed such as `long_fail`, `no_new_mut_0`, `new_train_val_tracks_success`).

19. Caveats found while reading the code

Things that behave differently from what a reader would probably expect. Items marked **(tested)** were reproduced by running copies of the project code; the others follow directly from the code.

#	Where	What happens
1	Crossover.single_cut, multi_cut, single_cut_weighted, multi_cut_weighted; Mutation...connection_based	**(tested)** They join DNA with `+`. With numpy DNA from `random_bits` this is element-wise addition, not concatenation → `ValueError: operands could not be broadcast`. Use them only with list-producing initializers, or switch to `np.concatenate`.
2	Diploid route	**(tested)** `decode_rates` raises TypeError for Double_DNA (reads `.DNA`, which is None); bestDNA writing and genotypic diversity do the same.
3	COMPUTATION in main.py	Only Car.py (validation, visual route) uses it; the headless training simulation has its own built-in feed-forward. (The 4-px vs 1-px sensor mismatch listed here before was fixed on 2026-10-10.)
4	FastDNA.GENE_LEN = 32	Marker length other than 8 is not handled consistently by the fast decoder and crossover.
5	graph_islands, graph, _draw_island_diversity	**(tested)** `graph_islands` and `graph` call `twinx()` and `fig.legend()` on every render, `_draw_island_diversity` calls `fig.legend()`, and old ones are never removed (5 renders → 6 axes, 5 legends). The right axis looks bold and drawing gets slower over a long run. Create the twin axis once, or clear the figure.
6	Mean mutation rate on the graph	Mean of a skewed distribution; median ≈ 0.0001 while the mean is ~0.057 (§7.2).
7	Window size = 90 % of the display	Segments are scaled to the screen, but speed (4 px/frame), turn rate and sensor step are absolute. Runs on different monitors are not directly comparable.
8	Drawing mode	Numbering restarts at 1 and overwrites existing segment files.
9	cellular_encoding	Hidden ids are assigned in termination order, which can turn tree links into backward edges; reversing them can point them away from the outputs (§8.9).
10	test.py	Out of date with Car's signature and graphData's format (§18).
11	requirements.txt	numba is missing; without it the NumPy fallback runs silently (capped sensors, much slower).
12	Island vs single mode	Random injection is 0 % per island but 10 % in single mode; elitism and copying also differ (§14.5).
13	main.py comment on MIGRATION_SIZE	Says 'sent along each edge'; the code sends K per island, each to one random neighbour.
14	Resume	The checkpoint's generation is evaluated a second time and the graph history restarts in a new folder.
15	Validation as model selection	bestValDNA is chosen by validation score, so the validation track is used for model selection; an unbiased generalisation estimate needs a third, untouched test track (what test.py was for).
16	Changes of 2026-10-10	1-px sensors, normalised speed input and reversed backward edges change what every DNA does: results, bestValDNA files and checkpoints from before are not comparable with new runs.
17	Phenotypic diversity	Fixed 2026-10-10: `_edge_signature` now stores edges as evaluated — (lower id, higher id), no self-loops — so genes 9 → 2 and 2 → 9 are one edge. Diversity values of new runs are not directly comparable with older diversity graphs.

Appendix A — Recipes

Use a different encoding (example: matrix)

```
init = Initializer().init_single_DNA_one_chromosome(INPUTS, OUTPUTS, MARKER)
dec  = Decoder().decodes_single_DNA_one_chromosome(INPUTS, OUTPUTS, MARKER)
DNA_INITIALIZATION = init.matrix_connections            # produces a Python list
DNA_DECODER        = dec.matrix_connections
CROSSOVER          = Crossover().crossover_single_DNA_one_chromosome(_MARKER=MARKER).matrix_connections
MUTATION = MutationCombiner([
    Mutation().mutate_single_DNA_one_chromosome(_MARKER=MARKER).random_bit_flip,
    Mutation().mutate_single_DNA_one_chromosome(_MARKER=MARKER).matrix_connections,
]).mutate
```

The headless simulation works with any decoder because it only needs the connection list. Leave out insert/delete for fixed-layout encodings if you want to keep the layout aligned (they shift every later bit; bit flips do not).

Turn elitism on, or change the topology

```
ELITE_FRACTION = 0.02                       # 2 elites per island of 100
ISLAND_GRAPH = 'ring'                       # or 'complete', or an adjacency matrix:
ISLAND_GRAPH = [[0,1,0,1],                  # 4 islands: 0-1, 0-3, 1-2, 2-3
                [1,0,1,0],
                [0,1,0,1],
                [1,0,1,0]]                  # NUM_SUBPOPULATIONS must be 4
```

Add, insert or disable a track segment

Put an image of the window size into `assets/train/` (wall = pure green, road = anything else, road touching the left-centre and right-centre). Name `5.5.png` to place it between 5 and 6; rename to `x5.5.png` to disable it.

Re-simulate a saved driver without the GA

```
dna = [int(c) for c in open("evolutionOutput/<run>/bestValDNA.txt").read().strip()]
d = Single_DNA_one_chromosome(); d.DNA = np.array(dna, dtype=np.uint8)
nn = Decoder().decodes_single_DNA_one_chromosome(INPUTS, OUTPUTS, MARKER).connection_based_markers(d)
W = build_weight_tensors([nn], 16)
# then call simulate_numba(W, ...) with a collision map built like EvolutionEngine.assembleTrack,
# or create one Car(...) with the current signature and call car.update(t) in a loop
```

Appendix B — Polish UI terms

Polish	English	Where
Ewolucja pojazdów	evolution of vehicles	window title
Generacja / Generacje	generation / generations	title, console, graph x-axis
Żywych	alive (cars)	title bar while drawing
Czas	time (frames / 120)	title bar
Obecny najlepszy wynik generacji	current best score of the generation	title bar
Walidacja najlepszego	validation of the best	title bar during validation
Przystosowanie	fitness	graph y-axis
Wyspa, sąsiedzi	island, neighbours	console
Różnorodność	diversity	console, diversity graph
Graf populacji – gen. X (migracja za Y gen.)	population graph – gen. X (migration in Y gen.)	island topology title

Polish	English	Where
Najlepsza sieć — trening / walidacja	best network — training / validation	network images
czerwony = waga +, niebieski = waga −, grubość ~ |waga|	red = positive weight, blue = negative, thickness ~ |weight|	network images
Rysuj segment … Enter=zapisz i następny, Esc=zakończ	draw segment … Enter = save and next, Esc = finish	drawing mode

Appendix C — Planned experiments: 17 neurons and Gray decoding

17 neurons with % 17

What has to change

- **main.py:** `OUTPUTS = [13, 14, 15, 16]` (INPUTS stay 0–5). `NR_OF_NEURONS = OUTPUTS[-1] + 1` is derived in the Decoder, the Initializer and Computation, so every id becomes byte % 17 and the hidden neurons become 6–12 (7 instead of 6).
- **Nothing else for the active configuration.** Checked: the marker decoder then produces ids up to 16, `build_weight_tensors` returns 17 × 17 tensors (it gets the size from Computation), NumbaSim takes the size from W, Car.py / Computation work with any ids, `fixed_topology` grows to 70 connections (6 × 7 + 7 × 4), and `cellular_division` / `cellular_encoding` use % NR_OF_NEURONS and the hidden pool automatically.
- `test.py` has its own copy of INPUTS / OUTPUTS — update it if you revive it.
- Do not resume or compare with older runs: every id in every genome changes meaning.

Encoding with a hard-coded 16	What breaks	Places to change
matrix_connections	fixed 16 × 16 = 256-bit matrix: neuron 16 (brake) can never be reached (checked: max id 15)	decoder (range(16), i·16 + j, RATE_BITS + 256), initializer (`[0]*256`, `src*16 + tgt`), crossover and mutation (RATE_BITS + 256), DNA.combine (256) → 17 × 17 = 289 bits
triangular_matrix_connections	120 bits and the 15·i index formula are for 16 neurons	same places; 17 neurons need 136 bits and k = 16i − i(i−1)/2 + (j − i − 1)
grammar_matrix	built on 4 × 4 → 16 rows of 16 bits; 17 is prime and does not fit the 4-way rewriting	needs a redesign (e.g. grow a 32 × 32 matrix and crop to 17 × 17)

What it changes

With % 16 the id is the low nibble of the byte and the high nibble is neutral: 8 of the 24 payload bits of every gene (the two high nibbles) can mutate without any effect. With % 17 a byte 16·h + l gives (l − h) mod 17, because 16 ≡ −1 (mod 17). Every bit now matters, and in a structured way:

Bit of the id byte	0	1	2	3	4	5	6	7
Effect of setting it (mod 17)	+1	+2	+4	+8	−1	−2	−4	−8

- So it is not chaotic in the sense of random jumps: one flip moves the id by at most 8 steps around a 17-cycle. But the neutral space disappears, so far more mutations change the network (less neutral drift; the self-adapted mutation rate will probably settle even lower).
- Ids are labels, so a ±1 step can change the meaning completely (input 5 = speed → hidden 6).
- 256 = 15 × 17 + 1, so id 0 (the 50° right ray) gets 16 of the 256 byte values and every other id 15: a slight bias.
- A random gene connects an input directly to an output with probability 6·4 / 17^2 = 8.3 % (now 9.4 %); usable, i.e. not a self-loop: 16/17 = 94.1 % (now 93.75 %).

- The dense feed-forward costs O(neurons²) per car and frame: 17^2 / 16^2 = +13 % network work.

Gray decoding

Read every integer stored in the DNA as a Gray code: `value = gray_to_binary(bits_to_int(bits))`. utils.py already has `binary_to_gray` / `gray_to_binary` (unused), and ideas.py has a prototype (`DNATools`). Gray code is a permutation of 0…255, so random genomes decode to exactly the same distribution — only **which values are one bit flip apart** changes.

What has to change

1. One switch in utils.py, e.g. `USE_GRAY = True`, `decode_uint(bits)` = gray_to_binary(bits_to_int(bits)) when on, and `encode_uint(n, k)` = int_to_bits(binary_to_gray(n), k) when on.
2. **Decoding:** use decode_uint instead of bits_to_int in `decode_rates` (both 16-bit rates) and in every decoder: connection_based (source, target, weight), the matrix / triangular / fixed / grammar weight bytes and grammar symbols, cellular_division ids and weights, cellular_encoding op-codes and pool weights.
3. **FastDNA._decode_markers** (Numba) builds the bytes inline; pass a `use_gray` flag and decode with a running XOR (code below) for source, target and weight.
4. **Encoding** (only where integers are written into DNA): `utils.connectionToDNA` (used by the connection_based and marker initializers) and the int_to_bits calls in `Initializer.cellular_division` → encode_uint. `random_bits` needs nothing.
5. Crossover and mutation need no change (they work on bits); the `decode_rates` call inside every crossover picks up the switch.
6. Old DNA files and checkpoints decode to different networks.

```
s = 0; b = 0
for k in range(8):
    b ^= dna[idx + k]          # Gray -> binary, most significant bit first
    s = (s << 1) | b
```

What it changes

Value	Binary (now)	Gray
weight byte	127 → 128 needs all 8 bits flipped (a Hamming cliff); flipping the top bit shifts the weight by ±5.02	every ±1 step is one flip, so small weight tuning is always one mutation away; flipping the top bit maps v → 255 − v, i.e. it exactly **negates** the weight (checked for all 256 values)
neuron id (% 16)	high nibble neutral, low-nibble flips change the id by ±1, 2, 4, 8	binary bits are XORs of all Gray bits above them, so nothing is neutral: flipping any high-nibble bit maps id → 15 − id (checked), e.g. input 2 → output 13
16-bit rates	top bit worth 0.5; insertions shift the field	no cliffs between neighbouring rates, but a top-bit flip reflects the rate: 0.0001 → 0.9999 (checked) — worse than binary
cellular_encoding op-codes	000 = E, 001 = 1, … as in the table of §8.9	the 3-bit codes map to different operators

Tip

Gray mainly helps weight tuning. For the mutation rate a reflected or doubled value is destructive, so if you switch the rates to Gray, also bound them, e.g. a log mapping such as rate = 10^(−5 + 4·v / 65535) (range 1e-5 … 0.1) or protect bits 0–31 from insert/delete.

Gray and % 17 combine without problems (Gray is a permutation, so the per-id counts stay 16 / 15). Both remove the neutral high-nibble bits, by different mechanisms, so change one at a time if you want to measure the effect.